

Runtime Observability for Heterogeneous Attention Memory

Fanzhe Wei

Metask Lab

whyer1@gmail.com

Li Liu

Metask Lab

muriel092611@gmail.com

Ziyang Wang

Logistics Science and Technology Innovation Integrated
Development Research Center of Shaanxi Logistics Group,
Xi'an Jiaotong University, Xi'an, China
and

School of Management,
Xi'an Jiaotong University, Xi'an, China

wang-zy13@tsinghua.org.cn

Chenyu Wang

Metask Lab

cnados@gmail.com

Abstract

Modern models no longer keep a plain KV cache: latent caches, learned sparse selectors and recurrent states each carry the model’s memory in a different form, and each fails differently under compression. We give a runtime observability contract for all four memory classes, built from three operators (UPDATE/SELECT/READ) and instantiated on six model configurations across five architecture families. Three results. *Typed composition*: contracts carry their error metric as a type, so a cross-metric chain is rejected at construction—this check refuted our own first composed chain—and every claim carries a machine-decided tier (certified, partially certified, or empirical), with composition inheriting the weakest tier. *An executable risk ledger*: replayed over 12.4M entry reads and run under eight-way concurrency with per-request budgets and fail-closed identity attribution, the ledger holds its risk budget with zero violations, and a fused always-on probe observes a declared one-layer subset under CUDA graphs inside the serving noise floor. *Diagnosis on a served stack*: applied to DeepSeek-V4 with a packed compressed-KV prototype, the same machinery localizes a silent corruption to a precise structural boundary—exact in the eviction-free, identity-isolated regime, with every observed failure under eviction or slot reuse—through a machine-adjudicated discrimination campaign whose calculus rejected two of our own confounded inferences along the way. All artifacts, guards, and the Lean development are released at <https://github.com/metask-ai/witprobe-attention-memory>; every number in this paper regenerates from the shipped artifacts by one command.

1 Introduction

On 2026-06-25 a silent corruption in a hierarchical cache took down agentic traffic on one of our production hosts. Nothing in the serving stack was in a position to notice: the cache was structurally valid, the model kept generating, and the failure surfaced only as degraded task success far downstream. That incident is the motivation for this paper. A compressed or shared attention memory is state that the system trusts without verifying, and the trust is currently blind.

The obvious response—“check the KV cache”—no longer names a single object. Contemporary architectures keep their attention memory in at least four different forms: dense per-token key/value tensors; a latent cache decompressed on read; a learned selector that picks a sparse subset of entries;

and a recurrent state carried forward rather than stored. They fail differently, they admit different guarantees, and a bound proved for one says nothing about another.

We make four contributions.

A contract for attention memory. We write any attention memory as three operators—update, select, read—and give each stage a local contract $e_{\text{out}} \leq a e_{\text{in}} + b$ with an explicit failure budget δ (§2). Supporting a new architecture becomes a proof obligation rather than a new bespoke tool.

Composition with a machine-decidable strength. Local contracts compose by affine composition and a union bound. The contribution is not the algebra, which is elementary, but that the platform *computes* the composition from live probe readings and labels every number certified, partially certified or empirical. Composition inherits the weakest tier, so “end-to-end certified” is a statement the system refuses to print unless every stage earns it. This caught a live error in our own chain: an early version used the *true* residual as the storage bound, a quantity that requires an uncompressed reference and therefore cannot be computed at runtime at all.

Cross-architecture evidence with an always-on cost. We instantiate the platform on six models across five architecture families (§4, §5), report per-stage readings with their measurement regimes, and measure the cost of leaving the probes on under concurrency.

Diagnosis on a served stack. Pointed at a served DeepSeek-V4 stack with a packed compressed-KV prototype, the same machinery ran a certificate-guided format exploration, localized a silent corruption to a precise structural boundary through a machine-adjudicated discrimination campaign—whose calculus rejected two of our own confounded inferences—and served the packed path end to end under a six-path serving-tax taxonomy (§6).

Artifacts. The measurement mathematics, the acceptance gates, every frozen run artifact behind every number in this paper, the figure and claim generators with their guards, and the complete Lean development (including the experiment-adjudication calculus) are released as witprobe-attention-memory (<https://github.com/metask-ai/witprobe-attention-memory>) under Apache-2.0; every number regenerates from the shipped artifacts by one command, and the claim guard fails the build on any mismatch. The architecture-specific probe-injection implementation is not part of the release (§7).

Terminology. A *contract* is a per-stage guarantee template whose error metric is part of its type; instantiated on a model, it is monitored through a *witness*—the locally computable quantity the probes read. Every emitted number carries a *tier*: *certified* (sound bound, not saturated), *partially certified* (bound on a decidable subset), or *empirical* (observation, no bound); composition inherits the weakest tier (§2.3). The *ledger* is the executable request-level account that composes per-stage failure budgets into one risk verdict (§5.5). A *measurement regime* is the serving condition under which a number was obtained—serial or concurrent, CUDA graphs on or off, eviction-free or under eviction; numbers are comparable only within a regime, and every reading below is annotated with its regime.

2 Attention Memory Contracts

2.1 Three operators

Write the memory at step t as m_t , and let

$$m_{t+1} = U_t(m_t, x_t) \quad \text{update / write} \quad (1)$$

$$I_t = S_t(q_t, m_t) \quad \text{select (possibly the identity)} \quad (2)$$

$$y_t = R_t(q_t, m_t, I_t) \quad \text{read} \quad (3)$$

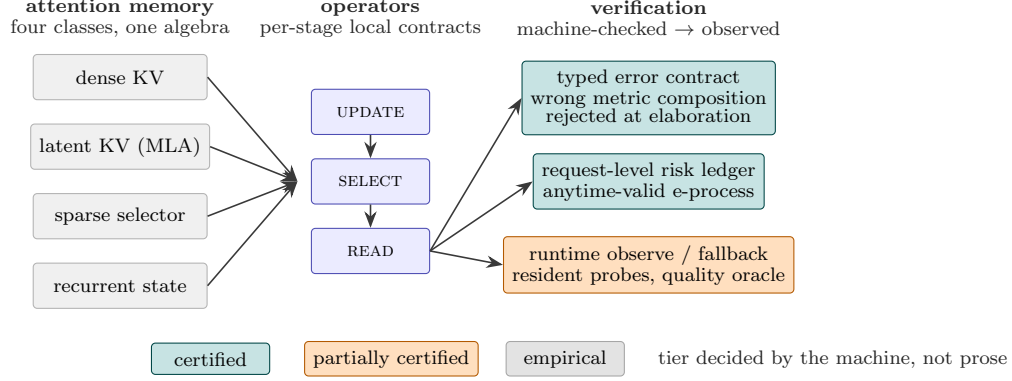


Figure 1: **WitCert in one view.** Four attention-memory classes, three operators, typed per-stage contracts, a request-level risk ledger, and fail-closed runtime observation. Colors mark certification tiers, decided by machine-checked artifacts (§2.3), not narrative.

Compression or corruption replaces m_t by $\hat{m}_t$; the goal is to bound $\|y_t - \hat{y}_t\|$ from locally computable witnesses. The four memory objects are instances: a dense KV cache appends and selects everything; a latent cache appends a compressed representation and reconstructs on read; a sparse selector adds a non-trivial S ; a recurrent state makes U a contraction rather than an append.

2.2 Local contracts and composition

Definition 1 (Contract). *A stage exposes (a, b, δ) with $e_{\text{out}} \leq a e_{\text{in}} + b$ holding except with probability δ .*

Series composition and the request-level budget are

$$(a_2, b_2) \circ (a_1, b_1) = (a_2 a_1, a_2 b_1 + b_2), \quad \sum_i \delta_i \leq \delta_{\text{req}}. \quad (4)$$

Both are elementary. What is not elementary in practice is that each a , b and δ must be *computable from quantities the serving stack actually has*, and that the resulting number must carry an honest strength label.

2.3 Three tiers, decided by the machine

- **certified** — a sound upper bound exists and is not saturated;
- **partially certified** — the bound holds only on a decidable subset, e.g. rank-prefix stability holds only for rows whose margin exceeds 2ε ;
- **empirical** — an observation with no bound attached.

Every metric the platform emits is registered against one tier together with the reason it earns that tier, and an unregistered metric is a hard error rather than a silent demotion. Composition takes the weakest tier of its stages. This is what makes the distinction operational rather than rhetorical: a chain containing one empirical stage cannot be reported as certified, whatever the accompanying prose says.

3 Local Contracts

3.1 Pooling is an attenuator, not an error source

Block-pooled architectures form one stored entry from B consecutive tokens with gate weights w , $\sum_j w_j = 1$, $w \geq 0$. If the tokens carry errors Δ_j then $\Delta c = \sum_j w_j \Delta_j$ and:

Lemma 1 (Pooling, two forms). *With $e_{\text{in}} := (\sum_j \|\Delta_j\|^2)^{1/2}$, $\|\Delta c\| \leq \|w\|_2 e_{\text{in}}$ by Cauchy-Schwarz, so $a = \|w\|_2 \leq 1$. With $e_{\text{in}} := \max_j \|\Delta_j\|$, convexity gives $a = 1$ with no assumption at all.*

Both forms must be reported: the ℓ_2 form is tighter but presumes an ℓ_2 aggregation of the incoming error, and quoting it alone suggests the attenuation is unconditional. Note that pooling is part of U and acts on *uncompressed* inputs, so it is not an error source; it governs how token-level errors upstream of it propagate. Placing it inside the baseline chain, as we initially did, yields a bound that is both looser and semantically wrong.

3.2 Selection is additive, not amplifying

Lemma 2 (Rank-prefix selection). *Regard the attention distribution as supported on the full key set and zero outside the selected set. With I the original selection and $\hat{I}$ the perturbed one,*

$$\text{TV}(p, \hat{p}) \leq \frac{1}{2} \left(\underbrace{\sum_{i \in I \setminus \hat{I}} p_i}_{m_{\text{out}}} + \underbrace{\sum_{i \in \hat{I} \setminus I} \hat{p}_i}_{m_{\text{in}}} \right) + \frac{1}{2} \sum_{i \in I \cap \hat{I}} |p_i - \hat{p}_i|, \quad (5)$$

hence $(a_S, b_S) = (1, \frac{1}{2}(m_{\text{out}} + m_{\text{in}}))$.

The selection stage therefore *adds* a bias equal to half the swapped attention mass and does not amplify upstream error. This reframes a negative result: asking whether the entire top- k set is preserved is the wrong question, because strict set certification is almost never attainable (§5), whereas the swapped mass is small and directly measurable.

3.3 A sampling sentinel with a detection-latency guarantee

A sentinel that draws r uniform slots *with replacement* per round (matching the implementation's `randint`) from M slots of which B are corrupted misses everything in one round, and over n independent rounds, with probability

$$q = \left(1 - \frac{B}{M}\right)^r, \quad \Pr[\text{missed after } n \text{ rounds}] = q^n. \quad (6)$$

(Sampling without replacement would give the smaller hypergeometric q ; modelling the implementation one actually runs is the point.)

Lemma 3 (Detection latency). *For target confidence $1 - \delta$, detection of at least one corrupted slot is guaranteed within $n(\delta) = \lceil \log \delta / \log q \rceil$ rounds.*

This converts an anecdotal demo into a monitoring guarantee with a tunable budget, and it explains an observation that would otherwise look like a capability limit (§5).

4 Platform

The implementation separates four concerns: a registry declaring where to inject for each architecture; adapters that extract the tensors and build the validity mask; meters holding the mathematics, which are architecture-independent; and a runtime that keys every accumulator by (owner, layer, path) and always emits a coverage record. Injection is anchored on exact upstream source blocks with an asserted hit count, and reverts surgically so that two adapters may share a file.

Acceptance is a fixed order—coverage, then magnitude, then soundness—and a failure at any step invalidates the numbers that follow. This is not a style preference. All three checks correspond to failure modes we hit in which the experiment terminated normally and produced plausible output: a probe whose per-layer state overwrote itself, a probe that read uninitialised device memory beyond a kernel’s valid range, and a probe attached to a write path the model never takes. Each produced a clean exit and a wrong number.

The platform currently registers six adapters over eleven injection points, covering the four memory objects.

5 Evaluation

5.1 Setup

All serving experiments run on a single node with 8×H200 (143 GB) GPUs on sglang 0.5.13.post1. The large-model experiments serve DeepSeek-V4-Flash-FP8 at tp=8/ep=8 with a 65,536-token context; overhead and graph-safety experiments use Qwen2.5-7B-Instruct on one GPU, where a serving stack restarts in minutes rather than tens of minutes. Regression runs for smaller models use a separate RTX 4090 machine (sglang 0.5.9). Probes are injected into unmodified serving code by a line-anchored source injector with hit-count assertions, byte-identical restore verified against SHA-256 baselines, and a fixed acceptance order—coverage, then magnitude sanity, then soundness self-checks—that must pass before any number is read. Unless stated otherwise probes sample one call in 8 with a 256-row cap per call, and probe measurements that need host-side I/O run with CUDA graphs disabled; the graph-enabled configuration is evaluated separately in §5.8. Every reported number is frozen from a probe artifact by a canon generator, and a claim guard fails the build if the manuscript and the artifacts disagree; the guard checks numeric strings, not semantics, so measurement regimes travel with the numbers in the artifacts themselves.

5.2 Coverage

Table 1 summarises the coverage: **8 rows / 6 models** across five families; two models were onboarded without writing any model-specific code (DeepSeek-V2-Lite needed only the first step of the onboarding recipe).

5.3 Per-stage readings

Latent memory. On the small proxy the latent meter is validated against per-cell ground truth over 4,032 cells with 0/4,032 Tier-A violations, and the dithered Tier-B coverage is 98.9% at $\tau = 0.2$. In production, GLM-5.2’s own fp8 storage residual is 0.93% of the latent norm, and our secondary int8 quantisation of the same latents measures 0.551% over all 78 layers. Latent norms are strongly heterogeneous across depth—a factor of 493× between the largest and smallest—which is the empirical basis for treating layers non-uniformly.

Model	Family	Probe class	Layers	Key reading
Qwen2.5-7B	GQA	KV	28/28	residual 0.80%, conserv. $3.25\times$
Llama-3.1-8B	GQA	KV	32/32	residual 1.02%, conserv. $3.77\times$
DeepSeek-V2-Lite	MLA	LatentKV	27/27	residual 0.877%, conserv. $2.69\times$
GLM-5.2-W4AFP8	MLA+DSA	LatentKV	78/78	residual 0.551%, conserv. $2.95\times$
GLM-5.2-W4AFP8	MLA+DSA	SparseSelector	21/21	top-1 certified 99.26%
V4-Flash-FP8	MLA+entry	LatentKV	43/43	residual 0.628%, conserv. $3.51\times$
V4-Flash-FP8	MLA+entry	SparseSelector	21/21	top-1 certified 99.40%
Kimi-Linear-48B	KDA+MLA	RecurrentState	20/20	$\bar{a} \in [0.530, 0.936]$

Table 1: **Coverage.** Eight probe rows over six models and five architecture families, all past the acceptance gate (coverage, magnitude, soundness); failing rows are not reported.

Sparse selection. Strict certification of the entire selected set is essentially unattainable: 0.05% on GLM’s token-level top-2048. The rank-prefix view is the opposite: top-1 stability is certified for 99.26% of rows on GLM and 99.40% on V4’s page-level top-512, with zero flips inside the certified region. The two are not directly comparable—one selects tokens and the other pages—so only the qualitative shape transfers.

Pooling. V4-Flash uses two pooling granularities in the same model, block 4 and block 128, over 21 and 20 layers respectively; since $1/\text{block}$ differs by $32\times$, raw concentration statistics cannot be compared across them and we report normalised quantities. The normalised concentration at block 4 is 0.1551, i.e. close to uniform.

Recurrent state. We report observation only. The per-layer mean contraction is $\bar{a} \in [0.530, 0.936]$, all below one, but the mean is the wrong statistic: long-run behaviour is governed by $\sum_t \log a_t$, and by Jensen’s inequality

$$\mathbb{E}[\log a] \leq \log \bar{a}. \quad (7)$$

Measured, $\mathbb{E}[\log a]$ has median -0.5912 against $\log \bar{a} = -0.2282$, so the mean form *understates* the contraction by $2.6\times$; the corresponding error half-life is 1.17 steps rather than 3.04. A residual gap remains: between 0.0 and 28.7% of samples per layer have $a \geq 0.999$, and our aggregate does not retain channel identity, so an aggregate $\mathbb{E}[\log a] < 0$ does not imply per-channel contraction. We therefore do not claim a certificate here.

5.4 Composition: the type checker rejected our own chain

Our first composed number added the storage stage’s bound to the selection stage’s b_S by (4) and reported the result. That composition is illegal, and an earlier version of this paper contained it. The storage bound is a *relative witness* on cache entries—a dimensionless ratio—while b_S is total variation on an attention distribution, a probability mass; $a_2 b_1 + b_2$ presumes b_1 is already denominated in the output metric of stage two, and here it is not. We caught this by making the metric part of the contract’s *type*: each contract declares an input and an output metric, composition is defined only when they coincide, and the same rule is enforced twice—at runtime, where the checker rejects the chain construction, and in Lean, where the ill-typed composition fails elaboration and the failure itself is pinned as a build assertion. Neither the tier system nor the numeric claim guard had caught it; both operate on values, and the values were individually correct. The retracted numbers appear nowhere else in this paper, and the guard now forbids them.

The repaired chain crosses between metrics through two proved bridges. The score bridge is

Cauchy–Schwarz: a perturbation Δk of a cache entry moves the attention score by at most

$$|\Delta(\text{scale} \cdot \langle q, k \rangle)| \leq \text{scale} \cdot \|q\| \cdot \|\Delta k\|. \quad (8)$$

Its coefficient is a runtime quantity, so the probe now collects it: across 43 layers and 8 ranks of V4-Flash, $\text{scale} \cdot \max\|q\| = 1.0034$ —queries pass through RMSNorm, so the bridge barely amplifies, and the bound is dominated by the witness W itself (per-layer max up to 0.700). The distribution bridge instantiates the machine-checked e-form theorem of the companion certificate paper [19] (there proved for per-token bounds c_t , here specialized to a uniform ε): a score perturbation of at most ε moves the softmax distribution by at most

$$\text{TV} \leq \frac{1}{2}(e^{2\varepsilon} - 1) \quad (9)$$

in total variation. Both directions of enforcement carry the theorem name in the contract’s proof field.

Review of the repaired chain caught a second mislabel, and we keep it on the record because it is the failure mode the type system cannot see: a hand-written metric annotation that is simply wrong. Our selection bound b_S is computed from the softmax of *indexer* logits, so its unit is selector-score mass, not attention mass; we had typed it as attention-TV by fiat. Retyping it correctly makes the checker refuse to add b_S to the storage term—the missing piece is an *empirical* stability bridge from selector scores to realised attention mass, which no amount of formalisation can conjure. The two are therefore reported separately and never summed.

What remains in the chain is the window-storage term, and its pairing is exact: the fused kernel that produces q also stores the window keys q attends to. It gives a per-layer attention-TV *bound* of 0.786 under the all-rank worst-case form (sampled-max coefficients; 29/43 layers below the trivial 1.0) and a *diagnostic reading* of 0.647 (median) under mean coefficients—the latter has no bound semantics, since a mean coefficient is not an upper bound, and we no longer call it one—for the comparison in which window entries are perturbed and selected pages are treated as exact—the quantisation of the compressed pages themselves is not yet witnessed, and the softmax bridge needs the ℓ_∞ over *all* logits, so we state that scope rather than claim past it. Summed over all 43 layers the budget is 36.3—vacuous, and reported as such; the sum is an aggregate over distinct per-layer distributions, not an end-to-end output distance, and no bridge across depth is claimed.

Both scope restrictions were then discharged by measurement rather than by more formalism: with every term witnessed on the 21 sparse-selection layers the full chain gives a worst-case per-layer TV bound of 1.473 (median), and the missing selector-to-attention bridge is replaced by a paired measurement—the ratio of attention mass to selector-score mass is 0.87 (median; converted with the tail value 2.18)—so the selection term crosses into attention units at tier *empirical*, and the composed tier drops with it, automatically. The full accounting, including why these numbers are *larger* than the retracted ones (they pay the full price of soundness), is Appendix H.

5.5 An executable request-level ledger

The per-stage contracts become operational through a request-level risk ledger with three rules: empirical objects never enter the certified ledger (they either fall back or degrade the request’s verdict); deterministic certificates—runtime checks whose premises the machine can decide—enter at $\delta = 0$; and probabilistic certificates draw from a telescoping budget

$$\delta_i = \frac{\delta_{\text{req}}}{(i+1)(i+2)}, \quad \sum_{i \geq 0} \delta_i < \delta_{\text{req}}, \quad (10)$$

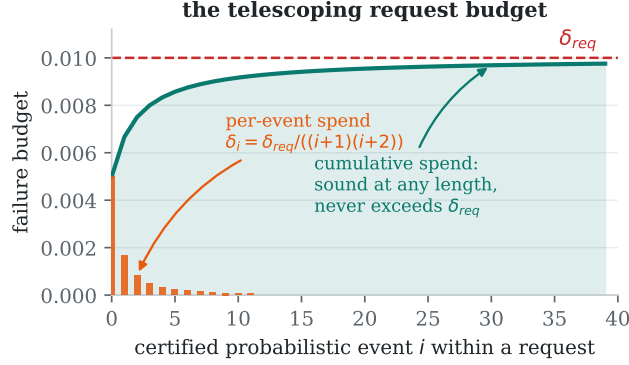


Figure 2: **The telescoping request budget.** Per-event spends $\delta_i = \delta_{req}/((i+1)(i+2))$ (orange) and their cumulative sum (teal) never reach δ_{req} (dashed): sound at unknown request length—the analytic content of `telescope_sum`, drawn at the implementation’s $\delta_{req} = 0.01$.

so the guarantee holds at every request length without knowing the length in advance. The budget rules are machine-checked theorems (`ledger_sound`, `telescope_sum`, `two_layer_budget_le`; Figure 2 shows the telescoping spend); the exclusion rule is not a theorem but an API discipline, enforced by the ledger implementation and locked by regression tests. The guarantee this machinery is built to deliver is the first tier of request-level claims: every read that uses a compressed result carries a valid local certificate, with failure probability at most δ_{req} . What this paper demonstrates is deliberately less, in two stages. An offline replay estimates the coverage of a prospective per-entry certified fallback policy over probe-sampled reads. An *online shadow ledger* then closes four of the six gaps between replay and execution: ledgers are instantiated per request with real request boundaries, every selected-page entry read at every decode step of every sparse-selection layer is checked online (198,163,408 entry events over 24 serial requests), and each request ends with its own verdict—median per-request retention 0.334 at the same threshold, with a hypothetical page-in of 5.4 GiB per request pricing what real fallback would move.

Certificate granularity decides whether any of this is useful. Our first replay certified whole decode steps against the step’s worst sampled entry; one large-witness entry then forced the entire step to fall back, and coverage was 0% at every useful threshold. Local fallback must be local: certifying *entries* makes the per-step TV bound hold by construction, and replaying 12.4M entry reads across the 21 sparse-selection layers gives the trade-off curve—80.1% compressed-entry retention at a vacuous per-step bound of 1.54, or $TV \leq 0.86$ at 29.3% retention (we say retention rather than certified coverage: quoting the retained fraction alone would hide that its per-step bound is vacuous). On the int8 re-quantisation witness the distribution is concentrated with no heavy tail to cut, so a non-vacuous bound and 80% retention cannot hold simultaneously: the witness, not the calculus, is the binding constraint. In this offline replay all certificates are deterministic and zero budget is spent ($\Sigma\delta = 0$); the write-time experiment below spends the probabilistic budget for real. The final two pieces touch the numeric path, and a controlled experiment lands both at write granularity—with one correction a stability control forced on our own earlier reading. Compression is made real by FP8 mantissa masking (a genuine coarsening of every stored window entry’s value grid, though the byte count is unchanged: this is precision coarsening, not physical compression), and the per-entry certificate decides, entry by entry, whether the coarsened bytes are written or the exact bytes are kept: fallback executes in the numeric path with zero page-in by construction. The policy accepts 48.0% of entries for masking (a policy acceptance rate, not a retained-bytes figure).

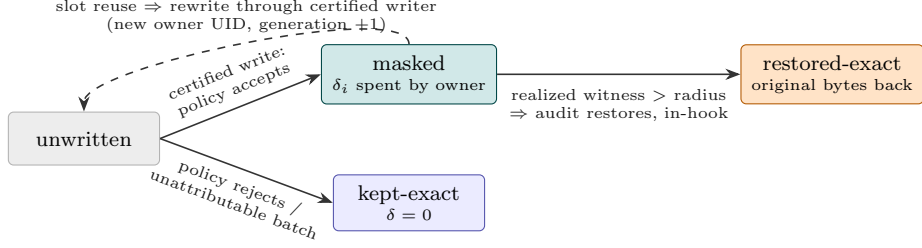


Figure 3: **The ledger’s slot-state machine.** Admission into *masked* is probabilistic (it spends the owner’s δ_i); enforcement is deterministic—a violating entry is restored to exact bytes in the same hook call, before any shared read. Reused slots re-enter only through the certified writer, with fresh owner and generation.

Probabilistic budget is spent with *pre-authorization* semantics: a prior radius authorizes an entry before any random bits are drawn, a write-side audit counts realized violations (0 of 147,208 authorized entries), and the price of authorizing before the draw is quantified rather than hidden ($6.2\times$ fewer entries than post-hoc gating accepts; mechanics in Appendix F).

The read-side per-request ledger and the write-side policy run in the *same* serving process over the same requests, and the loop now closes on the *same physical entries*: the certified write acts on the compressed-page pool—the buffer the read-side ledger dequantizes—and a per-slot status map (unwritten / masked / kept-exact) links every policy decision to its later reads. Over eight serial user requests, 3,936,205 of 7,750,144 read-side entry checks landed on slots the policy had masked, with zero reads of unwritten slots. Slot reuse is handled by *owner-tagged slot state* (with a per-rewrite generation counter): the compressed pool has no separate free path, so a reused slot is necessarily rewritten through the certified writer before it can be read again—the status map stays fresh by construction—and each write records the writing request’s UID. Under real reuse pressure (11,763 owner-change events from serial page recycling) the per-request closure stays exact: distinct prompts read zero foreign-owned entries, while a deliberately repeated prompt reads 94% of its entries from the prefix cache (974,386 of the run’s 1,063,892 foreign-owned reads; the remainder trace to server warmup batches that carry no request identity) and every one is attributed to its writing request rather than miscounted into the reader’s closure (write-side δ is charged to the owner; read-side checks are deterministic, so sharing does not disturb budget semantics). Four runtime invariants make shared-read semantics fail-closed rather than observational (Figure 3)—write-audit before any shared read, explicit slot state, on-the-spot byte restoration (admission is *probabilistic*, enforcement is *deterministic*), and once-per-write budget spend (**shared_read_sound**); the precise statements are Appendix F. Mid-request eviction workloads remain uncovered. Request identity is a never-reused monotone UID detected on the write path itself, so account boundaries coincide with true request boundaries; each request carries an independent telescoping budget (the eight user accounts each stay under budget, maximum spend $\Sigma\delta = 0.009982 \leq \delta_{\text{req}} = 0.01$), and the stochastic-rounding stream is per-(nonce, UID, layer, counter) and rank-invariant across the eight replicas (mechanics in Appendix F). Pre-draw authorization audited 0 violations in 120,661 authorized entries on this pool. Concurrency is covered by per-row identity: within a mixed batch every entry is attributed to its request via the compressor’s plan alignment (decode plans are batch-row-aligned; prefill plans are per-query-token, segmented by the prefix sum of extend lengths), each account keeps its own budget counter and random stream, and under eight simultaneously-submitted requests every account independently satisfies its budget (maximum spend $\Sigma\delta = 0.009984 \leq \delta_{\text{req}}$) with the owner ledger reporting *zero* foreign-owned reads across 8,228,864 checks. That owner

ledger doubles as a runtime guard on attribution itself: its first concurrent run flagged 3.4% foreign reads and traced them to a namespace collision between the two compressed-pool families—invisible under serial traffic, where a single active request masks ownership churn—which is precisely the class of accounting error a purely serial evaluation cannot surface. On output quality we report the control first: greedy decoding across two independent server restarts with *no* intervention already disagrees on 2 of 8 diverse prompts. Against that stability floor, the policy and dithered arms’ output differences from baseline are at or near the floor at $n=8$ and cannot be resolved; the mask-everything control still diverges grossly, so the intervention is real, and we describe the gated arms as *consistent with* a protective effect rather than demonstrating one. We use the word executable for exactly this scope—write-time policy with pre-authorized probabilistic budget, plus read-side per-request accounting in the same run—and quality claims stay at the eight-prompt directional level.

5.6 Upgrading the certified object

The witness being the binding constraint, we changed the certified object itself: from a worst-case norm to the exact conditional distribution of stochastic rounding. An offline adjudication over raw captured traces (queries, entry bytes and scales for all 43 layers; 64 rounding realizations per entry as ground truth) compared four certificate families against pre-registered gates. The centred bound—softmax is shift-invariant, so only the oscillation of the score error matters, and the two-point extremal argument gives the sharp constant

$$\text{TV} \leq \tanh(\text{osc}/4) \quad (11)$$

—moves non-vacuous layers from 0/43 to 43/43 on this object. Tightness is measured against the matching realized object per rounding realization (a review corrected our first metric, which divided a centred bound by an uncentred realization): the centred oscillation bound sits $5.2\times$ above realized oscillation, and the exact-MGF per-entry radius $2.1\times$ above realized $|\Delta s|$ under a joint δ/E budget, against $15.8\times$ for the current-form witness bound. Three of our own bugs were caught by the adjudicator’s soundness self-check (radii must never be beaten by realizations): a two-sided Chernoff combined with min instead of max; a deterministic shift from top-of-range clamping that is not random at all; and a zero-weight branch producing $\log 0$ that a clamp silently turned into fake perfection. One gate failed and is honoured: no family reaches $\tau=0.2$ retention at two-bit coarsening, so the heavy fused-kernel stage is skipped by the pre-registered rule.

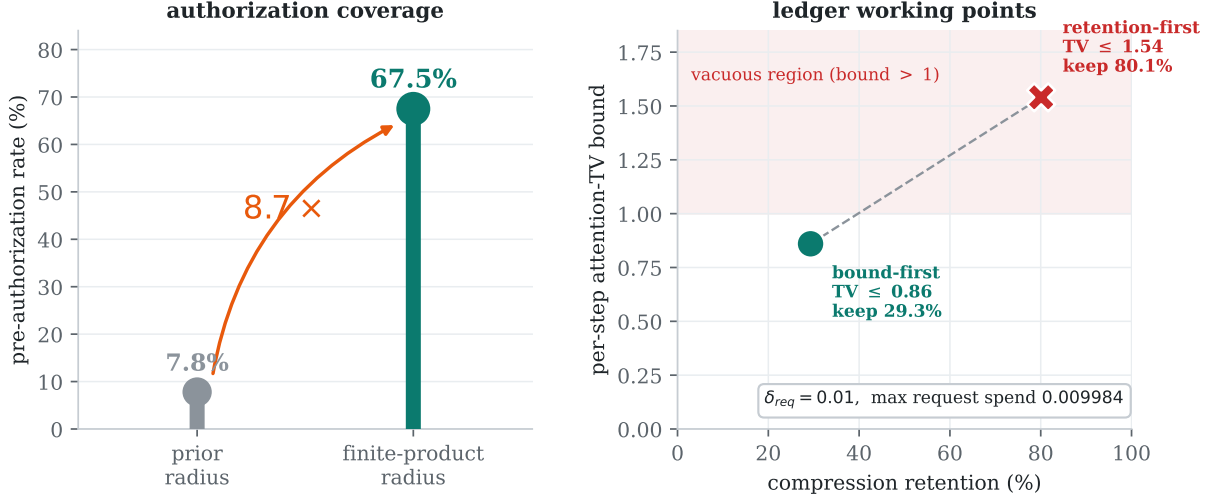
Deployed at write time—where the pre-quantization value is in hand, so the rounding position θ costs no metadata—true stochastic rounding (conditionally exact-zero mean) with a *variance-aware two-point bounded-difference radius*—exact two-point variance $\theta(1-\theta)\Delta^2$ and exact clamp shift, composed through Jensen and a Hoeffding tail. All three components of this radius are machine-checked on the finite product space with the constants the implementation uses—no measure theory anywhere, because the randomness is finitely many discrete draws:

$$\text{tail: } F'' = \frac{\theta(1-\theta)E_1E_2}{g^2} \leq \frac{1}{4} \quad \text{two-point Hoeffding, sub-Gaussian } \Delta^2/4 \quad (12)$$

$$\text{witness: } \text{bounded-difference MGF on the band witness} \quad \text{finite-}\Omega \text{ McDiarmid, by induction} \quad (13)$$

$$\text{mean: } \mathbb{E} \sum_b \|d_b\| \leq \sum_b \sqrt{\sum_c \theta(1-\theta)\Delta_c^2} \quad \text{recursive Jensen step} \quad (14)$$

The radius raises the pre-authorization rate at the same threshold from 7.8% to 67.5% ($8.7\times$), with



0 / 1, 276, 631 authorized entries violate their radius

Figure 4: **The certified object, quantitatively.** Left: the finite-product radius lifts pre-authorization coverage from 7.8% to 67.5% at the same threshold (0 of 1,276,631 authorized entries violate). Right: the ledger’s two working points—retention-first keeps 80.1% but its per-step bound (1.54) is vacuous; bound-first certifies $TV \leq 0.86$ at 29.3% retention. Max request spend $0.009984 \leq \delta_{req} = 0.01$.

0 of 1,276,631 authorized entries violating their radius. For the budget, a finite- Ω Ville inequality—provable by a three-line induction on hitting probabilities, with no measure theory, because the randomness is finitely many discrete draws—gives an anytime-valid e-process, and we now run one *on the same object as the ledger*: each authorized entry contributes

$$g_e = \exp(\lambda_e(W_e - m_e) - \lambda_e^2 C_e / 8), \quad \lambda_e = 4\kappa / \sqrt{C_e}, \quad \kappa = 0.5 \text{ pre-registered}, \quad (15)$$

built from its realized witness W_e against its pre-draw mean term m_e ; the supermartingale condition $\mathbb{E}[g_e | \mathcal{F}] \leq 1$ is itself machine-checked (it composes the McDiarmid MGF bound with the mean-term lemma). The sentinel runs at two scopes: per-request accounts, and a *global* process that never resets across requests—the continuously-running, whole-service form. Over 120,661 authorized-entry factors spanning all requests, the global process never approaches its crossing threshold:

$$\sup_t \log M_t = -0.82 \ll \ln(1/\delta_e) = 4.61 \quad (16)$$

(final value $-76,819$; we report the post-update maximum rather than the initialization value, which would overstate the evidence): an anytime-valid check that the stochastic-rounding radius assumptions are not contradicted by live traffic. Its semantics are model validation—a drift sentinel on the same entries at the same instants—complementing, not replacing, the per-entry union budget that authorization consumes. Separately, on the two-sided cumulative object with 33,407 events the e-process radius is $390\times$ tighter than summing telescoped per-event radii; the two certify *different failure events* (“any per-entry radius is beaten” versus the signed cumulative sum), and we report that magnitude comparison under that explicit caveat. The unfused write-path implementation costs $1.20\times$ in per-request wall clock (graphs-off regime, serial requests); the fused-kernel stage that would reduce it was skipped by the pre-registered gate, and physical bytes are unchanged throughout—precision coarsening is not physical compression.

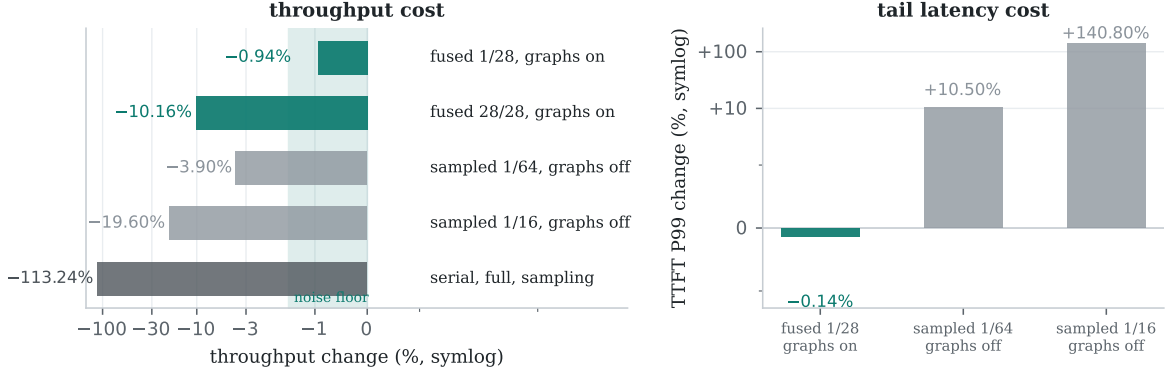


Figure 5: **Resident probe cost.** Under CUDA graphs the fused meter at one of 28 layers costs -0.94% throughput (-0.14% TTFT P99), inside the noise floor; full coverage (-10.16%) remains a diagnostic mode. Graphs-off sampling points shown for cross-regime comparison.

5.7 Corruption sentinel

Over 144 baseline verification passes we observed zero false alarms; this is not a false-positive rate of zero, and by

$$(1 - p)^n \leq e^{-np} \quad (17)$$

the one-sided 95% upper bound on the rate is 2.06%. In the injection experiment, 2 of 4 corrupted slots were located. Lemma 3 explains why this is expected rather than a limitation—with one correction this revision makes explicit: the implementation samples *with* replacement, so the model must too, and our earlier hypergeometric analysis was optimistic about the implementation it described. With replacement, an effective post-injection window of $n \approx 17$ rounds sampling 128 of 1024 slots gives a per-block detection probability of 0.881, and the probability of finding at most two of four is 0.072. Detection latency is a budget, not a capability: reaching 99% confidence for a single corrupted slot takes 148/74/37/19 rounds at $r = 32/64/128/256$.

5.8 Overhead

Three measurement regimes are necessary, and the always-on claim rests on the last. Serially, with one request in flight and CUDA graphs disabled, the probes cost $+113.24\%$ at full sampling—the worst-case amplification, with no other work to hide host-side cost behind. Under concurrency the picture separates by sampling rate: at one sample per 64 calls the cost is -3.9% throughput and $+10.5\%$ TTFT P99, both inside the per-setting measurement noise, whereas one sample per 16 calls costs -19.6% and $+140.8\%$, clearly outside it. Two methodological points follow. The noise floor must be computed per setting; taking a global maximum would classify the 16-sample degradation as noise. And tail latency, not throughput, is the criterion for always-on operation: the two settings are indistinguishable on throughput alone.

The third regime is graphs *enabled*, and it is the configuration that justifies the word always-on: instrumenting one declared layer with the fused Triton meter costs -0.94% throughput and -0.14% TTFT P99 against an identical probe-off baseline, both inside the per-concurrency noise floor, with replay-time accumulation and declared-versus-observed coverage verified by the acceptance gate. How this configuration was reached—including the failure of call-rate sampling under graph capture and the acceptance-gate hole it exposed—is Appendix C; full-coverage instrumentation under graphs remains a diagnostic mode at -10.16% .

Claim	Evidence	Strength	Not yet shown
Request-level risk budget	Lean-checked ledger + 8-request serving replay	certified	cross-layer error propagation
Eviction-free packed consumption	three independent 12-doc rounds, exact	empirical exactness	eviction / slot-reuse safety
Ring write/finalize/recycle chain	machine-adjudicated concurrent gate, zero violations	diagnostic chain	non-resident recovery
Low-overhead resident probe	graph-on serving benchmark	one layer of 28	full-coverage always-on

Table 2: **Claim boundaries.** What is asserted, on what evidence, at which tier, and what is explicitly not asserted; each row’s numbers appear in the text with their measurement regimes.

6 A served DeepSeek-V4 packed prototype: diagnosis to consumption

The machinery of §5 was built to observe; this section is what happened when it was pointed at a served DeepSeek-V4 stack carrying a packed compressed-KV prototype: a certificate-guided format exploration, a silent corruption localized to a structural boundary, the packed path served end to end, and a six-path serving-tax taxonomy—every verdict generated from run artifacts, not summarized by hand.

6.1 Case study: certificate-guided low-bit format exploration

A boundary-discovery exercise drove an aggressive low-bit format study on V4-Flash’s compressed pools (block-Hadamard INT6/INT4, a fused unpack kernel 3× faster than the production FP8 dequantizer, a real packed pool in serving); the instrument falsified two of our own design assumptions and scoped precisely what a served consumption claim would require. The full account is Appendix D; its open item—served packed consumption—is what §6.3 then closes and stresses.

6.2 Closing the boundary: generation staleness, not quantization

The scoped follow-up closed the boundary, and the closure is itself a result about what the type system must carry: a 95-point retrieval collapse that radix caching had been masking was discriminated down to *generation staleness*—slot reuse with no invalidation call anywhere in the codebase—rather than quantization (the quantizer’s intrinsic relative error is 0.0245, unreachable from the observed readings), and unconditional write-site invalidation restored accuracy with a two-sided causal confirmation (1.000 vs. 0.200 under 4-way concurrency, six machine-checked criteria). The full discrimination narrative is Appendix G.

Two lessons feed back into the contract layer. First, the *observation site* is part of a metric’s type. The write-site shadow check—mathematically identical to the decode-side one—had zero discriminative power for this failure (0.967 vs. 1.008 across fix/control arms) because the pool has two writers and the site compares a decode-time snapshot against a write-time source: a cross-generation comparison by construction. The same metric at the consumption point (what the kernel is about to read, sampled 1/32, before any refill) separates the arms by 36×: 0.0246—matching the 0.0245 intrinsic within half a percent, reproduced across six independent runs—against 0.882. A bridge between metrics must therefore check not only units but *where* the quantity is measured.

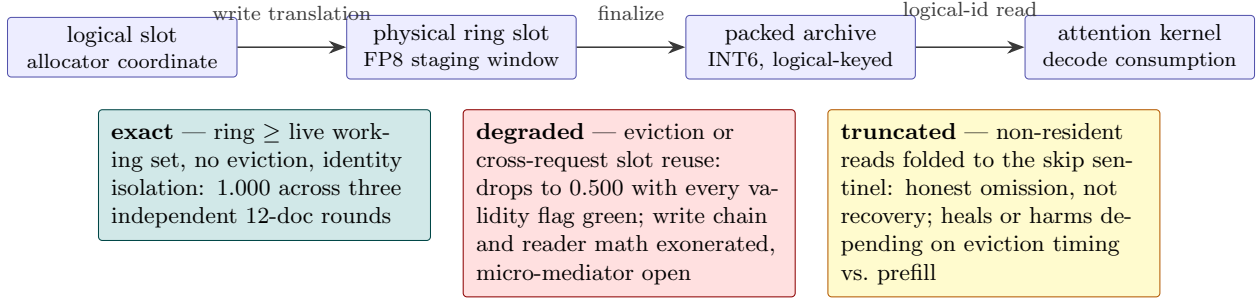


Figure 6: **Compressed-entry lifecycle and runtime regimes.** The exact regime is certified; degradation correlates with eviction and slot reuse while the write chain and audited readers are exonerated (mediator open, §6.3); the sentinel fold is a timing-dependent mitigation. The contract lesson: a replaced KV representation redefines coordinate, generation, and materialization semantics—all three belong in the runtime contract.

Second, (slot, generation) is a contract *premise*, not a logging field: the refinement obligation is that a consumed entry’s generation equals the generation at authorization, and write-site invalidation is its enforcement half.

The claims withheld above were then partially discharged—served consumption at parity, structural pool replacement quality-neutral, mechanism-level memory reduction with measured/accounting agreement of 1.7%—with one explicitly not discharged: the ring-staged configuration degraded retrieval while every fail-loud surface stayed green (the full discharge ledger, including the out-of-bounds addressing finding, is Appendix E). Both open items were subsequently discharged, and §6.3 records the closure and its measurement regime.

6.3 Serving the packed format: from diagnosis to consumption

The preceding subsection ended with two open claims—the ring lifecycle unproven, the $4\times$ pool untouched. The diagnostic ring chain and the physical pool-integration gates are now closed by machine-gated runs (per-*(layer, slot, epoch)* coverage with zero out-of-generation reads across eight ranks; an end-to-end packed INT6 replacement of the $4\times$ pool with measured 2,108 MiB/GPU HBM reduction against a tensor-derived byte model; in-graph packed consumption at exact retrieval parity), while eviction-safe production consumption remains open. What turned a quality regression into a structural boundary was a machine-adjudicated discrimination campaign—every comparison a controlled pair, every verdict a run artifact, two of our own confounded inferences rejected by the adjudication calculus on the way. Its shape: a per-pool dissociation pinned the harm to the $4\times$ ring’s eviction (dose-linked, reproduced across rounds); write-side forensics (16,942,020 bake-versus-store row checks, zero mismatches) exonerated the write chain; content oracles judged against calibrated repeat-pair noise floors proved the persistent stores and the entire prefill computation arm-invariant; and an intervention that folds non-resident reads to the kernel’s skip sentinel restores the eviction microcosm from 0.500 to 1.000 under matched eviction and call counts—honest truncation, effective but timing-sensitive, a mitigation rather than a repair. At twelve-document concurrency, eviction dose itself is scheduling-stochastic, and the truncated and untreated arms are consistent with one common dose–response relationship—no treatment shift is detectable at the present sample size (a quick regression reads +3.35pp with an interval spanning zero)—so the certified operating point is structural, not parametric (Figure 6, and quantitatively Figure 7: thirty-two runs from three series are consistent with one common dose–response relationship). The exclusion list is now long

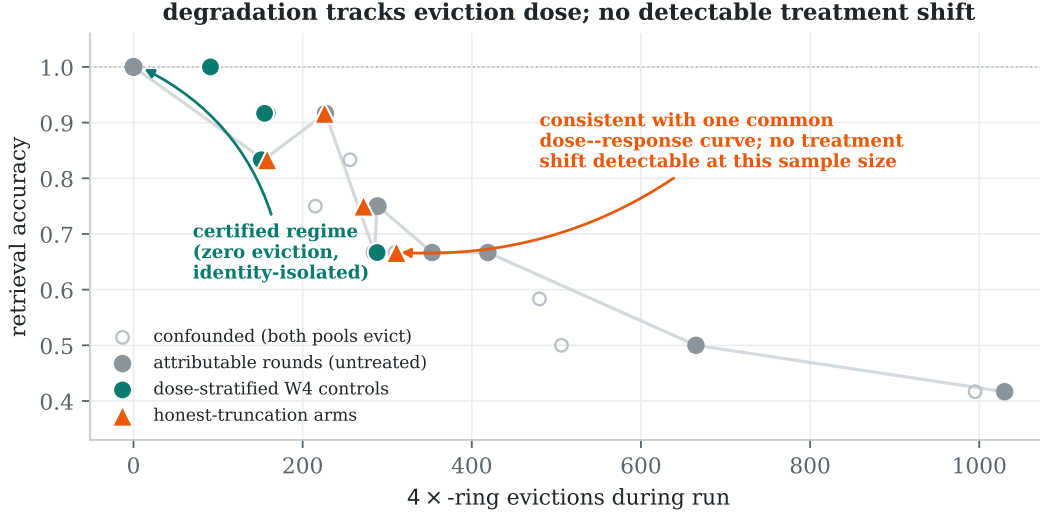


Figure 7: **Eviction dose governs quality.** Historical attributable rounds (gray; open = dose-confounded, context only), dose-stratified twelve-document controls (teal), and honest-truncation arms (orange) are consistent with one common dose-response relationship; no treatment shift is detectable at the present sample size. The zero-eviction cluster is the certified regime. Every point is read from its run artifact.

and two-sided: beyond the write chain and the persistent stores, a materialized reference reader—the instrument-verified dequantization path with reference attention mathematics, executed 1,430 times in a matched arm—reproduces the same 0.500, so the production kernel’s inline unpack and the reader mathematics are exonerated as well. What remains is a stable structural association—every observed failure occurs in an eviction or slot-reuse regime, while eviction alone does not force failure (one 91-eviction control run scores 1.000)—whose microscopic mediator is open, with the remaining candidate surface in selection metadata, index semantics, or their timing state; we record it as an open question rather than narrate beyond the evidence. The full round-by-round adjudication log, with every number read mechanically from run artifacts, is Appendix A. What is proven and production-relevant is unchanged: the zero-eviction, identity-isolated configuration is exact across three independent twelve-document rounds, and it is the only configuration this campaign certifies.

6.4 Path-contract evaluation

Bringing a replaced KV representation into a production stack is not one compatibility question but six, and the value of a runtime contract is that it answers each with a different failure type. We evaluated the packed stage against six serving paths and report a five-level taxonomy (Table 3)—PASS / FAIL-METHOD / BLOCKED-UPSTREAM / PARTIAL / NOT-MEASURED, of which four occur across these six paths—generated mechanically from the run artifacts rather than summarized by hand.

The matrix carries the per-dimension verdicts; prose here keeps only what each row *teaches* (per-path evidence and regimes in Appendix B). CUDA-graph serving is exact in the eviction-free regime and regime-limited beyond it (0.75 at twelve-document concurrency); radix prefix caching fails with the method reproducibly (0.667 vs. 1.000 native, two independent rounds), so the installer refuses the combination loudly; speculative decoding passes on a two-seed zero-eviction rerun (1.000 on both arms, both seeds) after the adjudication chain attributed its

Path	Capt.	Id/gen	Packed	Evict	Xfer	Qual.	Verdict
CUDA graphs	✓	✓	✓	△	—	△	PARTIAL
TP repeat	✓	✓	✓	—	—	✓	PASS
Radix cache	✓	×	—	×	—	×	FAIL-M
MTP (spec. decode)	✓	✓	✓	✓*	—	✓	PASS
PD disagg.	✓	✓	×	—	×	×	PARTIAL
HiSparse	—	—	—	—	—	—	BLOCKED

Table 3: **Path-contract matrix**, generated from run artifacts. Cells mark which contract layer each path violates ($\triangle$ = holds at a stated caliber; — = n/a or unmeasurable; * = zero-eviction caliber, machine-annotated); per-verdict evidence in §6.4.

historical residual to the eviction mechanism. Prefill/decode disaggregation yields the sharpest lesson: the native arm passes exactly (1.000; the needle lives in the prefill worker’s context, so a correct decode-side answer certifies a real KV transfer), while the packed arm returns 0.000 with *zero* transport or server errors—transfer completed, kernels ran, service stayed up—because the transfer engine moves bytes by the pool-layout contract that the packed stage redefines. A run can be structurally green and semantically wrong; that is precisely the gap a runtime quality oracle exists to close. The same oracle then sharpened the eviction-free claim itself: in a matched benchmark round in the working-set-ring regime, a retrieval battery run *after* a 32-request load scored 0.917 with zero evictions—the missed document answered with *another request’s* needle code, because freed logical slots were reused while their ring pages stayed resident, so the ring served the previous owner’s rows without any eviction. Zero eviction is therefore necessary but not sufficient; eviction and cross-request slot reuse are two triggers of one structural exposure; the sentinel-fold intervention removes the exposed reads at microcosm scale (a timing-sensitive mitigation, §6.3), and eviction-safe consumption with a verified mediator account remains future work. The remaining path (hierarchical sparsity) is blocked upstream: the unmodified baseline crashes identically with and without graphs, so the path is unmeasurable in this build; the crash trace is preserved as an incident artifact so the classification is auditable rather than asserted. On cost we claim nothing yet: the staged consumer serves at $0.884\times$ native throughput and $1.131\times$ cost per output token in its unoptimized materialize-per-call form, and the fused-kernel arm’s $0.238\times$ / $4.201\times$ figures include per-call host-side auditing scaffolding that production operation would remove; these are upper bounds on overhead, not a benefit statement. On quality in the same regime, a teacher-forced paired perplexity round (24 natural+code documents, 98,568 tokens, tp4-matched arms, with bidirectional path-activation evidence: the packed arm records 1,112/1,040 packed consumptions on the two pools; the base arm carries no packed switches, adapters, or consumption counters, with absence recorded in its manifest) reads 1.3496 vs. 1.3495 (-0.00% ; successive matched pairs read $+0.05\%$ and -0.002% , within run-to-run jitter): the packed consumption path is perplexity-neutral, so the cost story is not hiding a quality tax. The matched final-table round quantifies the unoptimized path end to end in the working-set-ring regime: 26.8 versus 173.4 output tokens/s against the native arm on a 32-request benchmark, with the enlarged rings erasing the memory saving in this regime—an honest cost statement, with the benefit claim still withheld until eviction-safe consumption is demonstrated.

7 Limitations

The sampled-probe overhead measurements of Section 5 are taken with CUDA graphs disabled, so their absolute latencies do not represent graph-enabled production and only the relative deltas are meaningful; the graph-enabled regime is measured separately (§5.8) and carries its own scope. The probe itself, however, does run under graph capture, and we state precisely what we verified, because “the server starts” is far too weak a test: graph replay executes no Python at all, so a probe whose accumulation quietly stopped would look identical to one that worked. Liveness under capture is verified by exact count arithmetic (host and device counters grow by precisely the predicted call and replay totals), and the eight silent-corruption trap classes this took—four on the probe side, four rediscovered on the packed write side, each invalidating capture rather than crashing—are catalogued with their regression tests in Appendix C. The path-contract matrix of §6.4 is likewise a bounded claim: one model, one serving stack version, small retrieval batteries, largely single-seed; it demonstrates the *taxonomy*—which paths fail, in which of five distinct ways—not production coverage, and no end-to-end cost benefit is claimed anywhere in this paper.

The always-on result of §5.8 (one declared layer, fused meter, $-0.94\%/ -0.14\%$, Figure 5) took two changes—replay-constant sampling decisions and a fused Triton meter $24.2\times$ cheaper per invocation ($9.3e - 07$ max relative error against the reference)—whose engineering account, including why naive sampling silently vanishes from captured graphs and how a lifetime-coverage gate passed on a dead probe, is Appendix C. We state the scope precisely: what is demonstrated is always-on observation of a *declared subset*, not of every layer. Full 28-layer coverage under graphs still costs -10.16% throughput even fused, which is far outside the noise floor, so contract readings that need every layer remain a diagnostic mode rather than a resident one. The storage term is now denominated in total variation with both window and compressed-page entries witnessed on the sparse-selection layers, and the selection term crosses metrics through a measured ratio; two gaps remain open, and we name them. The page side is now witnessed on every layer that has one: of the 43 layers, 21 select over $4\times$ -compressed pages, 20 attend densely over $128\times$ -compressed pages (no indexer, hence no selection term), and the first two are uncompressed by architecture; the coarse-compression pages turn out to carry a witness about half the size of the sparse-selection pages’ (0.523 vs. 1.045 max), and their median worst-case per-layer TV bound is 0.976 against the sparse-selection layers’ 1.339 : the harder-compressed layers are, counterintuitively, the easier ones to certify. And the per-layer TV bounds do not compose across layers—each layer’s attention is a different distribution, and we claim no inter-layer bridge—so the request-level number is an aggregate budget, and a vacuous one. The empirical bridge ratio is measured on truncation sets and applied to perturbation swap sets, a homogeneity assumption the tier system prices in by holding the composed chain at *empirical*. The recurrent-state aggregate does not retain channel identity, as noted. The sentinel guarantee assumes corrupted slots are independent of the sampling. Finally, six evaluated model configurations from one serving stack are not a substitute for independent replication. The release boundary is stated plainly: the public repository reproduces every number, figure and theorem from shipped artifacts and sources, while the architecture-specific injection implementation (hook anchors, adapters, kernels) remains private—serving-level reruns therefore require the platform, and the paper’s verifiable surface is the artifact-and-guard chain, not a rerun claim.

8 Related Work

Runtime certificates for compressed attention. Prior work bounds the distortion introduced by KV-cache quantisation—per-channel and outlier-aware schemes such as KIVI [1] and KVQuant [2]—and uses the bound to gate or repair at decode time. That line fixes the memory object—a dense post-RoPE KV cache—and derives one bound for it. Our concern is orthogonal: we do not propose a tighter bound for that object, we ask what remains true when the object itself changes, and what has to be supplied for a new object to be admitted.

Compression for latent and sparse architectures. Latent caches (MLA [3]), learned sparse selectors [4], pooled hierarchies and linear-attention states [5], and token-selection or eviction schemes (H2O [6], SnapKV [7], attention sinks [10], GEAR [11]) each report end-task scores computed offline against an uncompressed reference—exactly the quantity a serving system does not have; an early version of our own composition used that true residual as a bound before the tier registry rejected it as unavailable at runtime.

Systems observability. Production serving stacks such as vLLM [8] and SGLang [9] expose throughput, latency and memory, and hierarchical caches expose hit rates; none of these observes whether the retained state is still faithful. The incident that motivates §1 was invisible to all of them. The sentinel of Lemma 3 is deliberately in the shape systems people already use—a sampled check with a stated detection latency and a tunable budget—rather than a new guarantee requiring a new abstraction.

Anytime-valid inference and concentration. The per-entry radius is classical concentration [12, 13] deployed *prospectively* (declared before the draw, funded by a budget that telescopes soundly at unknown length), and the drift sentinel is an e-process [16, 14, 15]. Our contribution on this axis is not a new inequality but a serving-system instantiation in which every constant matches the implementation and the whole chain is machine-checked in Lean/mathlib [18] over a finite product space, with no measure-theoretic axioms beyond the standard three.

Runtime verification. Monitoring executions against formal specifications is a mature field [17]; the usual setting is discrete properties over event traces. Our monitors are numerical—error radii, budgets, supermartingales—and the “specification” is a probability statement whose side conditions (identity resolution, write-audit ordering, per-event accounting) are enforced fail-closed and surfaced as all-zero gate criteria. The concurrency guard that exposed a slot-namespace collision is, in runtime-verification terms, a monitored invariant doing exactly what monitors are for.

Error propagation. The composition rule (4) is affine composition with a union bound, and the recurrent case is a discrete Grönwall inequality; we claim no novelty in either. The contribution is that heterogeneous memory stages are expressed in one form, that the coefficients are computed from live telemetry rather than assumed, and that the strength of the composed statement is decided by the machine.

9 Conclusion

Heterogeneous attention memory breaks the implicit assumption behind existing KV-cache certificates, including our own companion meter [19]: that there is one memory object to bound. We proposed typed observability contracts—per-stage error bounds whose metric is part of the type, composed only when metrics match, with proof obligations carried by name and tiers decided by the machine—and instantiated them across latent caches, sparse selectors, pooled hierarchies and recurrent states on production serving stacks. The type discipline did real work: it rejected our

own first composed bound, forced two bridge theorems and one paired measurement into existence, and priced every unprovable step into an explicit tier instead of a hedged sentence. On the systems side, capture-time layer selection plus a fused meter make declared-subset observation affordable under CUDA graphs, and an entry-level certificate ledger with local fallback turns the per-stage bounds into a request-level risk budget—sound at unknown length, with empirical objects excluded by construction, executed so far at write-time policy granularity with read-side shadow accounting. Applied to a served DeepSeek-V4 stack, the same machinery turned a silent quality regression into a precise structural boundary—exact in the eviction-free, identity-isolated regime, every observed failure in an eviction or slot-reuse regime—with the microscopic mediator honestly recorded as open after a machine-adjudicated campaign exhausted every instrumented channel; the contract lesson, that a replaced memory representation redefines coordinate, generation and materialization semantics, is the paper’s most transferable systems finding. What the calculus still cannot do is cross layers: per-layer distributions are distinct objects, and no inter-layer bridge is claimed. That boundary is not a rhetorical limitation; it is the next theorem the type system is waiting for.

References

- [1] Z. Liu et al. KIVI: A tuning-free asymmetric 2bit quantization for KV cache. *ICML*, 2024.
- [2] C. Hooper et al. KVQuant: Towards 10 million context length LLM inference with KV cache quantization. *NeurIPS*, 2024.
- [3] DeepSeek-AI. DeepSeek-V2: A strong, economical, and efficient mixture-of-experts language model. arXiv:2405.04434, 2024.
- [4] J. Yuan et al. Native sparse attention: Hardware-aligned and natively trainable sparse attention. arXiv:2502.11089, 2025.
- [5] S. Yang et al. Gated linear attention transformers with hardware-efficient training. *ICML*, 2024.
- [6] Z. Zhang et al. H2O: Heavy-hitter oracle for efficient generative inference of large language models. *NeurIPS*, 2023.
- [7] Y. Li et al. SnapKV: LLM knows what you are looking for before generation. *NeurIPS*, 2024.
- [8] W. Kwon et al. Efficient memory management for large language model serving with PagedAttention. *SOSP*, 2023.
- [9] L. Zheng et al. SGLang: Efficient execution of structured language model programs. *NeurIPS*, 2024.
- [10] G. Xiao et al. Efficient streaming language models with attention sinks. *ICLR*, 2024.
- [11] H. Kang et al. GEAR: An efficient KV cache compression recipe for near-lossless generative inference of LLM. arXiv:2403.05527, 2024.
- [12] W. Hoeffding. Probability inequalities for sums of bounded random variables. *J. Amer. Statist. Assoc.*, 58:13–30, 1963.
- [13] C. McDiarmid. On the method of bounded differences. In *Surveys in Combinatorics*, 148–188. Cambridge Univ. Press, 1989.
- [14] J. Ville. *Étude critique de la notion de collectif*. Gauthier-Villars, 1939.
- [15] S. R. Howard, A. Ramdas, J. McAuliffe, J. Sekhon. Time-uniform Chernoff bounds via nonnegative supermartingales. *Probability Surveys*, 17:257–317, 2020.
- [16] A. Ramdas, P. Grünwald, V. Vovk, G. Shafer. Game-theoretic statistics and safe anytime-valid inference. *Statistical Science*, 38(4):576–601, 2023.

- [17] M. Leucker, C. Schallhart. A brief account of runtime verification. *J. Logic and Algebraic Programming*, 78(5):293–303, 2009.
- [18] The mathlib Community. The Lean mathematical library. *CPP*, 2020.
- [19] F. Wei, L. Liu, Z. Wang, C. Wang. WitCert: sound runtime risk observability and gating for KV-cache quantization. Companion paper, arXiv:2607.28699; artifact: <https://github.com/metask-ai/witcert-kv-certificates>.

A The discrimination campaign, round by round

This appendix preserves the full adjudication log summarized in §6.3. Every number is read mechanically from the corresponding run artifact; nothing here is narrated from memory.

First, the ring write/finalize/recycle chain passed a machine-adjudicated concurrent gate: a per-*(layer, logical-slot, write-epoch)* coverage ledger checked every decoded slot reference against the generation that produced it, across both arms and all eight ranks under full-ring pressure, with zero out-of-generation reads—a diagnostic-chain claim (ring→finalize→scratch-decode book-keeping), not a claim about the packed read path itself. Second, packed data is now *consumed* in serving rather than merely audited: a non-fused reference consumer classifies every referenced slot as ring-resident, packed-final, or fail-loud, and under an evict-invalidates-coverage semantics it re-materialized 26,440 rows from packed INT4 while both arms held exact retrieval parity (the three-way identity *resident + packed + failed + skip = total* holding per rank, tolerance zero). Third, the 4× pool—the 32× memory bulk deferred above—was replaced end-to-end with a packed INT6 truth source under the same machinery: the formal gate (nine criteria including a client-abort perturbation arm) passed with measured HBM reduction of 2,108 MiB per GPU against a tensor-derived byte model (all inputs read from live storage sizes, not derived estimates), and the 128× single-layer staging variant measured a median 316 MiB per GPU net against its own byte model. Finally, the consumption chain was brought inside CUDA graphs: with both graph modes enabled, the packed-kernel wrapper recorded 5,576 packed invocations (2,720 INT4 from the 128× pool, 2,856 INT6 from the 4× pool) at exact retrieval parity on both arms—a capture-time and eager-path call count, since graph replay executes recorded kernels without re-entering host counters; that the replayed graphs contain the packed branch follows from these being the branches taken at capture. The scope of that last claim is deliberately narrow—six needle documents, one seed, one hardware configuration, radix caching off, fixed ring sizes. Of the kernel invocations, 272 fell back to the native path; pointer-classified attribution later showed every fallback call carries no compression cache at all (it is not a compressed-pool read), so the fallback count is a call-mix figure, not a coverage gap. The narrowness matters: a twelve-document probe of the same configuration (no speculation) subsequently scored 0.750 with mid-depth misses that also recur under speculative decoding—a systematic residual that the six-document battery had been masking. A nine-round controlled-ablation chain then localized the defect (Figure A.1), and we report the chain because each round eliminated a hypothesis by construction. A zero-eviction ring restores exactness under full four-way concurrency (1.000, against 0.833 for the same round’s small-ring control), and the serial run that first suggested a concurrency skew turns out to have evicted nothing at all—“serial is clean” was a confound, and the invariant across all rounds is that corruption appears exactly when pages of in-flight requests are evicted. A per-pool dissociation pins the pool: evicting only the 128× ring is exact (reproduced three times), evicting only the 4× ring degrades to 0.667. The write side is exonerated by row-level forensics: 16,942,020 bake-versus-store checks across both pools found zero ownership mismatches, and an in-graph counter of finalizations touching freed pages read zero. The consumer was then found by a two-step probe. Coordinate-domain instru-



Figure A.1: **Nine-round controlled ablation.** Green: no pool evicts (exact). Blue: only the 128 $\times$ ring evicts (exact in all three repetitions). Red: only the 4 $\times$ ring evicts (degraded, with run-to-run drift under identical configuration). Corruption tracks 4 $\times$ -ring eviction dose; every point is read from its run artifact.

mentation showed that the compressed-context (“extra”) reads address the ring-registered buffers in cache-wide coordinates—observed indices reach 66 $\times$ the 4 $\times$ ring’s row count—and a contribution ablation that zeroes this path’s top- k lengths collapses retrieval to 0.000 against a 0.667 in-band control (the arms differ in downstream eviction counts, so this is a necessary-channel localization, not a dose-matched effect size): the reads are *essential*, and their data is whatever eviction leaves behind, which is why corruption tracks eviction dose rather than any timing discipline (two fence designs and an aged-reuse quarantine, all mechanically effective, all failed to help). The in-graph claim above is therefore a six-document, eviction-free-regime claim; a ring sized to the live working set is the demonstrated clean configuration. A prefill-side materialization adapter was then trialled by forcing the sparse-prefill branch, whose dequant anchor carries the translation-and-materialization shims: the corrected reads executed at scale (8,023,680 row translations, zero orphans) yet quality did not recover under eviction—correct addressing of the *prefill* phase is not the harmful mediator. The consumer was finally pinned on the decode side by closing the state space. A content oracle over the 4 $\times$ pool’s written entries (an L^1 -norm ordered-multiset metric, judged only against a same-configuration repeat pair’s calibrated noise floor) found the corrupt and healthy arms’ persistent contents statistically identical (median relative difference 0.0026 against a 0.0021 floor); the same instrument over the sliding-window pool—whose rows are a function of the prefill residual stream—likewise read at its floor (median 0.0016 against a 0.0010 floor), so the entire prefill computation is arm-invariant and the harm is confined to decode-time consumption. The read-side coordinate exposure is then arithmetic: compressed-context reads index the backing stores by *logical* slot (the read-side translation counter reads zero across all rounds, and the observed index bound exceeds the small ring’s physical capacity), while ring writes land at *translated physical* slots. An intervention isolates an effective lever: folding reads of non-resident (evicted) and out-of-range entries to the kernel’s skip sentinel—honest truncation rather than silent

consumption—restores the eviction microcosm from 0.500 to 1.000 against a knob-isolated control with matched eviction and call counts, the cleanest premise set in the adjudication chain. Scope is stated with the full honesty the campaign earned. The truncation lever is timing-sensitive: when eviction overlaps the prefill window the same truncation costs the needle’s context and the microcosm fails at 0.500, so this is a mitigation, not a repair. At the twelve-document concurrent scale a four-pair dose-stratified design plus a control-variance baseline shows eviction dose itself is scheduling-stochastic (a same-configuration control pair spans 0.667–1.000), and at matched dose the truncated and untreated arms sit on one dose–response curve: truncation and corruption are quality-equivalent harms on this retrieval task. Deepest of all, after nine further instrumented rounds—selection streams judged against a repeat-pair noise floor, served-content ledgers, archive-completeness audits at eviction time, key-space provenance—every measured quantity is arm-equal to its calibrated floor, the archive is complete and correctly keyed, and the quality split persists; the microscopic final mile of the harm mechanism is therefore recorded as an open question rather than narrated beyond the evidence. *Postscript (the rounds after the digest was frozen)*. Four further adjudications sharpened the exclusion list. A repeat of the intervention pair with early-landing evictions failed at 0.500 in both arms, establishing the truncation lever’s timing sensitivity. A key-space provenance oracle proved the packed archive logically keyed (readback matches the archive at INT6 quantization scale, 0.092 maximum relative difference, against 1.33 for the physical-key hypothesis). An eviction-time archive-completeness audit read 0.0216 mean relative difference (0.028 max, zero unwritten rows) between the evicted pages’ archive and the ring’s final values—the archive is complete. Finally, a materialized reference reader (the instrument-verified dequantization path plus reference attention mathematics, 1,430 calls in a matched arm) reproduced the control’s 0.500 exactly, exonerating the production kernel’s inline unpack and the reader mathematics; selection streams judged against a repeat-pair noise floor showed top- k scrambling is run-to-run nondeterminism, quality-irrelevant in clean pairs. Every one of these instruments narrowed the mediator’s possible location; none found it. The claim in the main text is calibrated accordingly.

B Per-path evidence and measurement regimes

CUDA-graph serving: in-graph packed consumption is exact at the six-document eviction-free regime (1.000/1.000); the twelve-document concurrent probe sits at 0.75 under the eviction mechanism of §6.3, so the path’s claim is regime-limited rather than unconditional. Repeat-run answer stability passes (1.000/1.000 across a same-config serial repeat pair; text-level determinism is explicitly not claimed, as the upstream deterministic-inference mode does not support this attention backend). Radix prefix caching fails reproducibly with the method—the second round after the read path was fully wired, excluding an instrumentation confound; lifting this requires prefix-reuse-aware generation semantics. Speculative decoding: its translation chain closed first (the fail-loud counters that once fired stay zero); an earlier cross-commit comparison suggesting nondeterminism was methodologically invalid, and a same-source rerun showed the misses recur without speculation; the two-seed rerun at the working-set-ring regime is zero-eviction, machine-annotated in the artifact—each round its own dual-arm control.

C Always-on probes under CUDA graphs: the engineering account

Under a graph-enabled run, the host-side call counter grew by 168, exactly the prefill calls (6 requests $\times$ 28 layers), while the device-side accumulators grew by 43176, exactly the 43008 decode

replays plus those same prefill calls; the counts were unchanged after an idle interval, ruling out a lagging flush. Since replay runs no Python, that growth can only come from probe operations captured into the graph. Four changes were necessary to get there, each worth stating because each is a silent-corruption trap rather than a crash: accumulators *including counters* must be device-resident tensors, since a host-side count would freeze at capture while the sums kept growing, inflating every mean; the sampling path must avoid RNG, which invalidates capture; flushing must be deferred outside the capture window, since reading accumulators back is a device synchronisation; and persistent accumulators must not be allocated during capture, because serving captures one graph per batch size and an accumulator allocated inside one graph’s memory pool is invalid from another. The packed-consumption runs of §6.3 rediscovered the same trap classes on the write side—a call-site argument evaluated before its guard, a lazily built device constant, a Python boolean scalar assigned into a device tensor, a data-dependent boolean-mask index—each invalidating capture rather than crashing, and each now a regression test.

Running under graphs is not the same as running always-on under graphs, and getting there took two changes rather than one. At full rate the naive cost is prohibitive: on a single-GPU Qwen2.5-7B serving stack with graphs enabled, throughput falls -52.8% and TTFT P99 rises $+138.6\%$ against an identical probe-off baseline. The obvious remedy—sample one call in N , which is what makes the probe affordable with graphs disabled—does not transfer, because the sampling test is a Python branch and Python runs only at capture. Its value at that instant decides whether a given graph contains probe operations at all, so N stops meaning “one call in N ” and starts meaning “these graphs, never those”. Measured at $N = 64$ under graphs, the accumulators grew by 0 across an entire workload while the overhead was correspondingly nil—a probe that is not in the graph is indeed free. Worse, that run *passed* our acceptance gate, which saw all 28 layers covered by samples left over from start-up, before graph capture began. Coverage over the run’s lifetime is not liveness during the run. We therefore added a device-side execution counter, incremented in place and hence captured into the graph, and a gate check requiring it to exceed the host-side call count; the element counts already collected cannot serve this purpose, since they cannot distinguish one call sampling many rows from many replays sampling one row each.

The first change is to make the sampling decision depend only on quantities that are constant across replays. A layer index is such a quantity, and selecting instrumented layers by index behaves as intended: at every stride we tried, replay accumulation is present and the observed layers match the declared set exactly. It is not sufficient on its own. Cost tracks the number of instrumented call sites and the number of kernels each site issues, and the two scale close to linearly and multiply: halving the quantiser work at a fixed two layers cuts the throughput cost by $1.72\times$, and going from two instrumented layers to one at a fixed single quantiser cuts it by a further $1.97\times$, reaching -2.26% throughput and $+1.32\%$ TTFT P99—still outside the noise floor. Sampling less cannot remove a per-execution constant, so the second change attacks the constant itself. We fuse the witness meter into a single Triton kernel that loads the sampled rows once, performs the per-band quantisation and both norms in registers, and folds the result into the accumulators, replacing several dozen kernel launches with one. It agrees with the reference implementation to a maximum relative error of $9.3e-07$ on every statistic, with violation counts and sample counts exactly equal—the violation count is a soundness self-check and is not allowed to be approximate—and a single invocation costs 12.3 us, or $24.2\times$ less than the unfused single-quantiser path. At equal, full 28-layer coverage the fused meter is $5.2\times$ cheaper than the unfused one.

With both changes, one configuration meets all three conditions we set in advance: instrumenting one layer in twenty-eight with the fused meter costs -0.94% throughput and -0.14% TTFT P99, both inside the per-concurrency noise floor, while replay accumulation is present and the observed coverage matches the declared single layer. We state the scope precisely: what is

demonstrated is always-on observation of *a declared subset*, not of every layer.

D Certificate-guided low-bit format exploration

As a boundary-discovery exercise—not a completed compression system—we used the instrument to drive an aggressive low-bit format study on V4-Flash’s compressed pools. The witness ladder selected block-Hadamard INT6 for the memory-dominant $4\times$ pool and INT4 for the $128\times$ pool on hidden requests (naive INT4 rejected: 2% coverage, $15\times$ the INT8 control’s score-shift proxy), a fused unpack kernel measured $3\times$ *faster* than the production FP8 dequantizer (bit-exact, tensor-core inverse Hadamard), and a real 360 B/token packed pool— $36.5 \rightarrow 22.5$ MiB for 64Ki tokens as a standalone allocation, a format-level figure we do not conflate with serving-process memory—ran in serving with 12,960 read-back checks, zero unwritten slots, and 10.2% mean relative error. The instrument then falsified two of our own design assumptions: entry slots evolve progressively, so write-time packing reads a non-final state (changing only the packing instant moved the discrepancy $18\times$); and the apparent Python read path is not the production consumer—decode reads FP8 pages inside the FlashMLA kernel, so three instrumented probes of the swap path never fired. We therefore claim format, kernel, pool, and write-side lifecycle as validated, and explicitly do *not* claim served INT4 consumption, HBM reduction in the serving process, throughput or quality effects: those require a packed-input FlashMLA variant, which the instrument has now precisely scoped. The residual $\text{rel}_{\max} \approx 4.9$ traces to re-reads of still-evolving slots, not quantization failure (the near-zero-norm hypothesis was tested and rejected).

E The staged-ring discharge ledger

The claims withheld above can now be partially discharged. Served consumption holds: with invalidation in place the packed pools serve the retrieval task at parity (1.000, 20/20, zero request errors). Structural pool replacement holds: swapping the $128\times$ pool to the packed object at the canonical address (twenty layers remapped, installation attested per rank) is quality-neutral (acc 1.000, decode-side fidelity unchanged). Physical memory reduction holds at mechanism level: shrinking the FP8 staging pool to a 257-page window ring freed 2,988 MiB of measured device memory across eight GPUs against an accounting estimate of 3,040 MiB—an aggregate agreement of 1.7%, though the per-GPU deltas are non-uniform (566 on rank 0, 346 elsewhere), so we claim aggregate agreement only. We state the measurement regime plainly: the $128\times$ pool is $\sim 0.3\%$ of process memory, and the $32\times$ -larger $4\times$ pool, where the same construction must be repeated for material savings, remains future work. One claim is explicitly *not* discharged: the ring-staged configuration initially degraded retrieval to 0.500 vs. 1.000 while every fail-loud surface stayed green—and the artifact carried no internal counters (an evidence-chain gap we record rather than paper over), so only the HTTP-level zero-error figure is attested. A scoped intervention later identified an addressing defect as a major cause (the production decode kernel indexing the 257-page ring with 20,413-page logical slot ids—an out-of-bounds silent read); at that stage, whether the write/finalize/recycle chain was itself loss-free remained open pending a port-separated re-run with poisoned-scratch instrumentation, and the stage stood as memory-mechanism evidence only.

F Ledger enforcement mechanics

Probabilistic budget is spent with *pre-authorization* semantics: before any random bits are drawn, a prior radius u_e (a mean term plus a $\sqrt{2\Sigma s_c^2 \ln(n/\delta_i)}$ tail computed from the stored exponents alone)

authorizes an entry only if u_e clears the threshold; unauthorized entries fall back without drawing. Each δ_i from the telescoping sequence is thereby bound to a concrete event—some authorized entry’s realized witness exceeding its pre-declared radius—and a write-side audit counts realized violations: 0 of 147,208 authorized entries violated their radius, validating the conditional-rate semantics at runtime ($\Sigma\delta = 0.0099997 < \delta_{\text{req}}$ over 33,407 events, with the moment-condition assumption logged with per-event counts). The price of authorizing before the draw is quantified rather than hidden: the prior radius authorizes $6.2\times$ fewer entries than post-hoc deterministic gating accepts (7.8% vs. 48.0%).

Four runtime invariants make this shared-read semantics fail-closed rather than observational: the write-time audit completes inside the same hook call, before any shared read; every slot carries an explicit state (unwritten / masked / kept-exact / restored-exact); a violating entry has its original bytes restored on the spot, so the served value respects the declared radius *deterministically*—“zero violations” is enforced, not merely observed; and an immutable write event spends its δ exactly once however many requests later read it, which is the content of a machine-checked lemma (`shared_read_sound`: the union bound ranges over write events, never over readers). Identity resolution itself is fail-closed: a batch whose per-entry request attribution cannot be established keeps exact bytes, spends no probabilistic budget, and increments a counter that the acceptance gate requires to be zero—the gate is a machine-decidable conjunction of twelve all-zero criteria (including missing-data checks: all eight replica snapshots present, sentinel factors nonempty) carried by the adjudication script’s exit code. The restore step upgrades the online semantics, and we state it precisely to avoid a reading we do not claim: admission is *probabilistic*, enforcement is *deterministic*. The request-level δ bounds the probability of *unexpected restoration events*—an authorized entry’s realized witness exceeding its pre-declared radius, forcing the audit to restore exact bytes—not the probability of serving an out-of-bound value: the served value respects its declared radius always, by the write-audit and restore invariants above. Shared readers inherit that deterministic guarantee and never multiply the originating write event’s risk.

Request identity is a monotone logical UID that is never reused: the boundary is detected *on the write path itself*, at the non-extend-to-extend transition that opens every request’s prefill, so it precedes that request’s first write and account boundaries coincide with true request boundaries (a fallback on sequence-length drops covers read-only deployments; the reuse-with-longer-prompt case the fallback alone would miss is pinned by a unit test). Each request carries an *independent* telescoping budget: the eight user-request accounts—reported separately from one server-warmup account—each stay under budget with maximum spend $\Sigma\delta = 0.009982 \leq \delta_{\text{req}} = 0.01$, and per-request event counters restart from $\delta_1 = \delta_{\text{req}}/2$ (the earlier implementation let one global counter starve later requests). The stochastic-rounding seed is the quadruple (server nonce, request UID, layer, event counter): distinct requests never share a random stream, so conditional randomness holds under adaptive traffic, while the quadruple is rank-invariant—the eight tensor-parallel replicas realize the *same* draw, agree field-for-field in their accounts, and δ is spent once per logical event, an empirically checked coupling rather than an assumption.

G The generation-staleness discrimination

This appendix gives the full evidence chain behind the boundary closure stated in §6.2. A long-context retrieval task (needle-in-haystack, exact match, matched five-arm control) first exposed a 95-point collapse that radix caching had been masking: the packed arm scored 0.800 with the cache on—a cache-hit figure, not a quality figure—and 0.050 with it off, while the FP8 baseline held 1.000 on both sides. A freshness discriminator (identical arms differing only in a re-pack-on-every-

read diagnostic, both write- and read-path execution counters required to be nonzero) recovered 0.050 vs. 1.000: generation consistency, not quantization, was the active hypothesis. Offline round-trips of the production pack path on real entries pinned the quantizer’s intrinsic relative error at 0.0245 (kernel, pool path, and overwrite all agreeing; maximum 0.0304; the RoPE segment exactly zero), so the $\text{rel} > 1$ values observed in serving were not achievable by the quantizer on this data at all—they were reads of a *previous generation*. The mechanism: slot reuse by the allocator with no invalidation call anywhere in the codebase, so a “written” flag conflated *has ever been packed* with *current generation*. Unconditional write-site invalidation (a boolean scatter plus a generation increment, placed before any sampling gate) restored task accuracy, and a second causal confirmation with the diagnostic disabled—arms differing only in the invalidation switch, with two-sided evidence (the fix arm’s invalidation counter positive, the control arm’s exactly zero)—returned INVALIDATION_CONFIRMED on six machine-checked criteria: 1.000 vs. 0.200 under 4-way concurrency.

H The repaired chain, fully accounted

This appendix accounts, term by term, for how both scope restrictions were discharged by measurement. The probe now witnesses the compressed-page entries the selector chooses (their witness is *larger* than the window’s, max 1.040 against 0.700, so treating pages as exact had been the optimistic side), and on the 21 sparse-selection layers where every term is witnessed the full chain gives a worst-case per-layer TV *bound* of 1.473 (median; 4/21 layers below 1.0) and a mean-coefficient *diagnostic reading* of 0.992 (median)—the diagnostic has no bound semantics. For the selection term, the missing selector-to-attention bridge is replaced by a paired measurement on identical candidate sets: the ratio of attention-mass omitted to selector-mass omitted is 0.87 (median; tail to 2.18, and we convert with the tail value), so b_S crosses into attention units as 0.00444 at tier *empirical*—and the composed tier drops to empirical with it, automatically. The same measurement separates two quantities our first analysis conflated: the top-512 truncation itself leaves 0.379 (median) of the hypothetical full-compressed attention mass unattended, but that is the architecture’s designed sparsity, not an error—the model’s ideal reference includes the truncation, and only perturbation-induced *changes* of the selection belong in the error chain. These numbers are larger than the retracted ones because they are denominated in a real metric and pay the full price of soundness: Cauchy–Schwarz assumes the perturbation aligns with the query, and the witness overstates the residual by construction. A bound that got smaller by adding incompatible units was not a bound. The self-checks remain 0 throughout, and the sensitivity ordering says where tightening pays: the witness W first, the alignment worst case second; $b_S = 0.00203$, in its own units, is not the bottleneck.

I Machine-checked theorem statements

Compiled, not transcribed. Every theorem name cited in the body (`ledger_sound`, `telescope_sum`, `shared_read_sound`, the bridge and e-process lemmas, ...) resolves to a statement below. This section is *compiled* from the Lean development shipped in the released repository (`witprobe-attention-memory`): `formal/Export.lean` walks the environment, prints each theorem’s *elaborated* type with Lean’s own pretty-printer, and lists the axioms each proof actually depends on via `collectAxioms`; non-ASCII symbols are mechanically transliterated for the pdf latex toolchain. Nothing here is transcribed by hand, so a statement that is not proved cannot appear. The contract calculus proves 14 *core* theorems—the results cited in the body—plus 143 supporting lemmas

listed compactly so the axiom audit covers the whole development; every proof depends only on `propext`, `Classical.choice` and `Quot.sound`, and none on `sorryAx`. The whole development re-verifies independently: clone <https://github.com/metask-ai/witprobe-attention-memory> and run `cd formal && bash check_all.sh`, which rebuilds every proof and re-runs the axiom audit from scratch; the `file:line` references below resolve in that repository.

conditional (adaptive) union bound over first-failure events *Name:*

`WitCert.Calculus.Budget.conditional_union_le`; *source:* `formal/WitCert/Contract.lean:143`;
axioms: `propext`, `Classical.choice`, `Quot.sound`.

```
forall {Omega : Type u_1} [inst : MeasurableSpace Omega] (mu : MeasureTheory.Measure Omega) (E
  : Nat -> Set Omega) (delta : Nat -> ENNReal),
(forall (i : Nat), mu (E i \ Union j in Finset.range i, E j) <= delta i) ->
  forall (n : Nat), mu (Union i in Finset.range n, E i) <= Sum i in Finset.range n,
    delta i
```

coverage confidence: $(1-p)^n$ $\models$ e^{-np} ; zero observed violations is not zero rate

Name: `WitCert.Calculus.Ledger.coverage_confidence`; *source:* `formal/WitCert/Ledger.lean:89`;
axioms: `propext`, `Classical.choice`, `Quot.sound`.

```
forall {p : R}, 0 <= p -> p <= 1 -> forall (n : Nat), (1 - p) ^ n <= Real.exp (-(n * p))
```

request-level ledger: conditional union bound + budget sequence, sound at every length *Name:* `WitCert.Calculus.Ledger.ledger_sound`; *source:* `formal/WitCert/Ledger.lean:66`; *axioms:* `propext`, `Classical.choice`, `Quot.sound`.

```
forall {Omega : Type u_1} [inst : MeasurableSpace Omega] (mu : MeasureTheory.Measure Omega) (E
  : Nat -> Set Omega) (deltaseq : Nat -> ENNReal)
(deltareq : ENNReal),
(forall (i : Nat), mu (E i \ Union j in Finset.range i, E j) <= deltaseq i) ->
  (forall (n : Nat), Sum i in Finset.range n, deltaseq i <= deltareq) -> forall (n :
    Nat), mu (Union i in Finset.range n, E i) <= deltareq
```

refinement interface: any `CertifiedEventStream` satisfies the ledger guarantee *Name:*

`WitCert.Calculus.Ledger.stream_sound`; *source:* `formal/WitCert/Ledger.lean:204`; *axioms:*
`propext`, `Classical.choice`, `Quot.sound`.

```
forall {Omega : Type u_2} [inst : MeasurableSpace Omega] {mu : MeasureTheory.Measure Omega}
(s : WitCert.Calculus.Ledger.CertifiedEventStream Omega mu) (n : Nat), mu (Union i in
  Finset.range n, s.E i) <= s.budget
```

telescoping budget weights $1/((i+1)(i+2))$: unknown-length budget sums to $1-1/(n+1)$ *Name:* `WitCert.Calculus.Ledger.telescope_sum`; *source:*

`formal/WitCert/Ledger.lean:37`; *axioms:* `propext`, `Classical.choice`, `Quot.sound`.

```
forall (n : Nat), Sum i in Finset.range n, WitCert.Calculus.Ledger.w i = 1 - 1 / (n + 1)
```

two-layer budget: deterministic events at $\delta=0$, probabilistic on telescoping counters *Name:* `WitCert.Calculus.Ledger.two_layer_budget_le`; *source:*

`formal/WitCert/Ledger.lean:144`; *axioms:* `propext`, `Classical.choice`, `Quot.sound`.

```
forall (delta : R),
  0 <= delta ->
    forall (f : Nat -> Option Nat),
      (forall (i k : Nat), f i = some k -> k = WitCert.Calculus.Ledger.probCount f i) ->
        forall (n : Nat),
          (Sum i in Finset.range n,
```

```

match f i with
| none => 0
| some k => ENNReal.ofReal (delta * WitCert.Calculus.Ledger.w k) <=
ENNReal.ofReal delta

```

unknown-length budgeting costs at most $\sqrt{2}$ in sub-Gaussian radius vs uniform split *Name:* WitCert.Calculus.Ledger.unknown_length_price; *source:*

formal/WitCert/Ledger.lean:112; *axioms:* propext, Classical.choice, Quot.sound.

```

forall {delta : ℝ}, 0 < delta -> delta <= 1 / 2 -> forall {N i : ℕ}, 1 <= i -> i <= N ->
Real.log (i * (i + 1) / delta) <= 2 * Real.log (N / delta)

```

e-process: product of conditionally mean- $\frac{1}{\delta}$ factors gives anytime-valid risk $\frac{1}{\delta}$ *Name:* WitCert.Calculus.Ville.eprocess_ville; *source:* formal/WitCert/Ville.lean:127; *axioms:* propext, Quot.sound, Classical.choice.

```

forall {sigma : Type u_1} [inst : Fintype sigma] (D : WitCert.Calculus.Ville.Draw sigma) (g :
List sigma -> sigma -> ℝ) {delta : ℝ},
0 < delta ->
delta <= 1 ->
(forall (h : List sigma) (x : sigma), 0 <= g h x) ->
(forall (h : List sigma), Sum x : sigma, D.p x * g h x <= 1) ->
forall (T : ℕ), WitCert.Calculus.Ville.hitProb D
(WitCert.Calculus.Ville.prodProcess g) (1 / delta) T [] <= delta

```

finite-Omega Ville inequality: $P(\sup M \frac{1}{\delta} \leq c) \leq M_0/c$, elementary induction, no measure theory *Name:* WitCert.Calculus.Ville.ville; *source:* formal/WitCert/Ville.lean:65; *axioms:* propext, Quot.sound, Classical.choice.

```

forall {sigma : Type u_1} [inst : Fintype sigma] (D : WitCert.Calculus.Ville.Draw sigma) (M :
List sigma -> ℝ) {c : ℝ},
0 < c ->
(forall (h : List sigma), 0 <= M h) ->
(forall (h : List sigma), Sum x : sigma, D.p x * M (x :: h) <= M h) ->
forall (T : ℕ) (h : List sigma), WitCert.Calculus.Ville.hitProb D M c T h <= M h / c

```

bridge contract semantics: cross-metric composition needs a proved bridge *Name:* WitCert.Calculus.bridge_sound; *source:* formal/WitCert/Contract.lean:118; *axioms:* propext, Classical.choice, Quot.sound.

```

forall {alpha : Type} {m m' : WitCert.Calculus.ErrMetric alpha} (B : WitCert.Calculus.Bridge m
m') (x x' : alpha),
m'.d x x' <= B.toContract.a * m.d x x' + B.toContract.b

```

typed contract composition: metrics must match by construction *Name:* WitCert.Calculus.comp_sound; *source:* formal/WitCert/Contract.lean:93; *axioms:* propext, Classical.choice, Quot.sound.

```

forall {alpha beta ? : Type} {malpha : WitCert.Calculus.ErrMetric alpha} {mbeta :
WitCert.Calculus.ErrMetric beta}
{m? : WitCert.Calculus.ErrMetric ?} (C2 : WitCert.Calculus.Contract mbeta m?) (C1 :
WitCert.Calculus.Contract malpha mbeta)
(x x' : alpha), m?.d (C2.exact (C1.exact x)) (C2.approx (C1.approx x')) <= C2.a * C1.a *
malpha.d x x' + (C2.a * C1.b + C2.b)

```

witness-to-score bridge: $\text{---Delta(scale} \cdot \mathbf{q} \cdot \mathbf{k})\text{---} \mathbf{i} = \text{scale} \cdot \text{---q---} \cdot \text{---Deltak---}$
(Cauchy-Schwarz) *Name:* WitCert.Calculus.score_bridge; *source:*
formal/WitCert/Bridges.lean:36; *axioms:* propext, Classical.choice, Quot.sound.

```
forall {d : Nat} (q k k' : Fin d -> R) (scale : R),
  0 <= scale ->
    |scale * Sum i : Fin d, q i * k i - scale * Sum i : Fin d, q i * k' i| <=
      scale * sqrt (Sum i : Fin d, q i ^ 2) * sqrt (Sum i : Fin d, (k i - k' i) ^ 2)
```

affine form of the score-to-TV bridge: $\text{TV } \mathbf{i} = e^{\{2\text{eps}0\}} \cdot \text{eps}$ for $\text{eps } \mathbf{i} = \text{eps}0$ *Name:*
WitCert.Calculus.softmax_tv_affine; *source:* formal/WitCert/Bridges.lean:160; *axioms:* propext,
Quot.sound, Classical.choice.

```
forall {iota : Type u_1} [inst : Fintype iota] [inst_1 : Nonempty iota] (s s' : iota -> R)
  (eps eps0 : R),
  0 <= eps ->
    eps <= eps0 ->
      (forall (t : iota), |s t - s' t| <= eps) ->
        WitCert.TV (WitCert.Calculus.softmax s') (WitCert.Calculus.softmax s) <= Real.exp (2 *
          eps0) * eps
```

score-to-TV bridge: $\text{---Deltas---} \inf \mathbf{i} = \text{eps} == \mathbf{i}$, $\text{TV } \mathbf{i} = 1/2(e^{\{2\text{eps}\}} - 1)$,
instantiating paper 1's tv_le_iform *Name:* WitCert.Calculus.softmax_tv_bridge; *source:*
formal/WitCert/Bridges.lean:93; *axioms:* propext, Quot.sound, Classical.choice.

```
forall {iota : Type u_1} [inst : Fintype iota] [inst_1 : Nonempty iota] (s s' : iota -> R)
  (eps : R),
  0 <= eps ->
    (forall (t : iota), |s t - s' t| <= eps) ->
      WitCert.TV (WitCert.Calculus.softmax s') (WitCert.Calculus.softmax s) <= 1 / 2 *
        (Real.exp eps ^ 2 - 1)
```

Supporting lemmas (axiom-audit completeness only) Apriori.TV_le_one
(WitCert/Apriori.lean:273); Apriori.TV_nonneg (WitCert/Apriori.lean:268);
Apriori.bdd.diff_of_lipschitz (WitCert/Apriori.lean:661); Apriori.bdd.diff_of_lipschitz_on
(WitCert/Apriori.lean:745); Apriori.bernoulli_mgf_le (WitCert/Apriori.lean:146);
Apriori.betting_factor_eprocess (WitCert/Apriori.lean:803); Apriori.betting_ucb_anytime
(WitCert/Apriori.lean:832); Apriori.condE_eq_prod_sum (WitCert/Apriori.lean:470);
Apriori.coord_mgf_le (WitCert/Apriori.lean:71); Apriori.doob_mgf_le
(WitCert/Apriori.lean:257); Apriori.doob_mgf_le_biased (WitCert/Apriori.lean:241);
Apriori.doob_mgf_prod_sum (WitCert/Apriori.lean:510);
Apriori.eprocess_factor_le_one_of_expected (WitCert/Apriori.lean:176); Apriori.iterComp_mem
(WitCert/Apriori.lean:757); Apriori.lip_comp (WitCert/Apriori.lean:672); Apriori.lip_iterComp
(WitCert/Apriori.lean:713); Apriori.lip_iterComp_on (WitCert/Apriori.lean:770);
Apriori.lip_residual (WitCert/Apriori.lean:692); Apriori.mgf_tensorize
(WitCert/Apriori.lean:40); Apriori.mgf_tensorize_le (WitCert/Apriori.lean:53);
Apriori.prod_measure_total (WitCert/Apriori.lean:28); Apriori.request_tail_of_expected_loss
(WitCert/Apriori.lean:204); Apriori.request_tail_of_served_tv (WitCert/Apriori.lean:311);
Apriori.request_tail_of_served_tv_doob_massweighted (WitCert/Apriori.lean:611);
Apriori.request_tail_of_served_tv_general (WitCert/Apriori.lean:376);
Apriori.request_tail_of_served_tv_massweighted (WitCert/Apriori.lean:420);
Apriori.request_tail_of_served_tv_topk (WitCert/Apriori.lean:531);
Apriori.served_tv_mean_le_cum_C (WitCert/Apriori.lean:105); BC_ge_subgaussian
(WitCert/Bridges.lean:568); BC_le_one (WitCert/Bridges.lean:522);
BatchEdit.applyAll_isSome_iff_allHit (WitCert/BatchEdit.lean:59);
BatchEdit.changed_but_not_all_applied (WitCert/BatchEdit.lean:81);
BatchEdit.one_hit_masks_many_misses (WitCert/BatchEdit.lean:91);

BatchEdit.overlap_makes_order_matter (WitCert/BatchEdit.lean:101);
 Budget.conditional_union_le_of_marginal (WitCert/Contract.lean:173);
 Conformal.average_does_not_give_per_history (WitCert/Conformal.lean:206);
 Conformal.conformal_exceedance_le (WitCert/Conformal.lean:97);
 Conformal.conformal_risk_union (WitCert/Conformal.lean:130);
 Conformal.exchangeability_needed (WitCert/Conformal.lean:220);
 Conformal.le_thr_of_not_strictMax (WitCert/Conformal.lean:113); Conformal.mem_strictMax
 (WitCert/Conformal.lean:69); Conformal.proble_one (WitCert/Conformal.lean:51);
 Conformal.prob_mono (WitCert/Conformal.lean:48); Conformal.prob_nonneg
 (WitCert/Conformal.lean:45); Conformal.prob_union_le (WitCert/Conformal.lean:56);
 Conformal.strictMax_pairwiseDisjoint (WitCert/Conformal.lean:75);
 Conformal.sum_prob_strictMax_le_one (WitCert/Conformal.lean:84);
 Conformal.ties_break_disjointness (WitCert/Conformal.lean:213);
 Conformal.total_risk_over_random_history (WitCert/Conformal.lean:169);
 CumLoss.admit_implies_realized_within (WitCert/CumLoss.lean:210); CumLoss.budget_floor
 (WitCert/CumLoss.lean:234); CumLoss.cumloss_admission (WitCert/CumLoss.lean:132);
 CumLoss.gating_conservative (WitCert/CumLoss.lean:195); CumLoss.hitProbP_eq_hitProb
 (WitCert/CumLoss.lean:86); CumLoss.hitProbP_le_one (WitCert/CumLoss.lean:39);
 CumLoss.hitProbP_mono (WitCert/CumLoss.lean:57); CumLoss.nonconservative_admit_can_exceed
 (WitCert/CumLoss.lean:223); CumLoss.prodProcess_exp (WitCert/CumLoss.lean:111);
 CumLoss.tailTerm_mono (WitCert/CumLoss.lean:184); Ledger.telescope_budget_le
 (WitCert/Ledger.lean:73); Ledger.telescope_sum_le_one (WitCert/Ledger.lean:51);
 Ledger.w_nonneg (WitCert/Ledger.lean:34); McDiarmid.BddDiffAt.c_nonneg
 (WitCert/McDiarmid.lean:72); McDiarmid.addF_nonneg (WitCert/McDiarmid.lean:443);
 McDiarmid.band_mean_le (WitCert/McDiarmid.lean:515); McDiarmid.condE_add
 (WitCert/McDiarmid.lean:387); McDiarmid.condE_addF (WitCert/McDiarmid.lean:452);
 McDiarmid.condE_bdd_diff (WitCert/McDiarmid.lean:79); McDiarmid.condE_congr
 (WitCert/McDiarmid.lean:408); McDiarmid.condE_const_mul (WitCert/McDiarmid.lean:51);
 McDiarmid.condE_exp_le (WitCert/McDiarmid.lean:201); McDiarmid.condE_finset_sum
 (WitCert/McDiarmid.lean:414); McDiarmid.condE_mono (WitCert/McDiarmid.lean:41);
 McDiarmid.condE_nonneg (WitCert/McDiarmid.lean:399); McDiarmid.condE_sq_le
 (WitCert/McDiarmid.lean:495); McDiarmid.eprocess_factor_mean_le_one
 (WitCert/McDiarmid.lean:556); McDiarmid.mcdiarmid (WitCert/McDiarmid.lean:310);
 McDiarmid.mcdiarmid_radius (WitCert/McDiarmid.lean:359); McDiarmid.sum_exp_le_of_mean_zero
 (WitCert/McDiarmid.lean:107); McDiarmid.witness_mean_le (WitCert/McDiarmid.lean:542);
 QuantityKind.sample_mean_below_threshold_not_certifying (WitCert/QuantityKind.lean:96);
 QuantityKind.scope_mismatch_blocks_pass (WitCert/QuantityKind.lean:102);
 QuantityKind.totalAlpha_nil (WitCert/QuantityKind.lean:113);
 QuantityKind.undetermined_consistent_with_both (WitCert/QuantityKind.lean:86);
 QuantityKind.undetermined_costs_nothing (WitCert/QuantityKind.lean:117);
 Radius.det_shift_sound (WitCert/Radius.lean:226); Radius.g_pos (WitCert/Radius.lean:47);
 Radius.g_zero (WitCert/Radius.lean:54); Radius.hasDerivAt_g (WitCert/Radius.lean:56);
 Radius.hasDerivAt_gd (WitCert/Radius.lean:66); Radius.hasDerivAt_phi
 (WitCert/Radius.lean:81); Radius.log_two_point_mgf_le (WitCert/Radius.lean:145);
 Radius.phi_deriv_le_quarter (WitCert/Radius.lean:100); Radius.phi_deriv_nonneg
 (WitCert/Radius.lean:94); Radius.phi_le_quarter_mul (WitCert/Radius.lean:111);
 Radius.phi_zero (WitCert/Radius.lean:79); Radius.quarter_mul_le_phi
 (WitCert/Radius.lean:128); Radius.second_deriv_num_eq (WitCert/Radius.lean:89);
 Radius.sq_sum_weighted_le (WitCert/Radius.lean:212); Radius.sr_mgf_le
 (WitCert/Radius.lean:190); Radius.two_point_mgf_le (WitCert/Radius.lean:184);
 Radius.two_point_variance (WitCert/Radius.lean:204); SharedRead.probE_mono_event
 (WitCert/SharedRead.lean:59); SharedRead.probE_union_le (WitCert/SharedRead.lean:48);
 SharedRead.shared_read_sound (WitCert/SharedRead.lean:79); Ville.hitProb_nonneg
 (WitCert/Ville.lean:45); Ville.prodProcess_supermartingale (WitCert/Ville.lean:106);

crossSum_comm (WitCert/Bridges.lean:716); crossSum_indep_meanzero_eq_zero
 (WitCert/Bridges.lean:784); crossSum_sum_left (WitCert/Bridges.lean:723);
 cumulative_output_tv (WitCert/Bridges.lean:418); exp_sub_one_le_mul_exp
 (WitCert/Bridges.lean:129); gate_threshold_for_sla (WitCert/Bridges.lean:206);
 linear_propagation_frobenius (WitCert/Bridges.lean:844); log_jensen_weighted
 (WitCert/Bridges.lean:904); massweighted_prodenv_bound (WitCert/Bridges.lean:950);
 massweighted_topk_bound (WitCert/Bridges.lean:920); one_sub_BC_eq (WitCert/Bridges.lean:505);
 pool_linf (WitCert/Contract.lean:189); propagated_second_moment_le
 (WitCert/Bridges.lean:853); rank_prefix_stable (WitCert/Contract.lean:208);
 residual_second_moment_indep (WitCert/Bridges.lean:821); residual_second_moment_le
 (WitCert/Bridges.lean:758); residual_second_moment_orthogonal (WitCert/Bridges.lean:737);
 score_perturbation_l2_le (WitCert/Bridges.lean:866); sentinel_rounds
 (WitCert/Contract.lean:225); served_tv_le_of_gate (WitCert/Bridges.lean:188);
 served_tv_le_of_gate_tanh (WitCert/Bridges.lean:390); served_tv_le_subgaussian
 (WitCert/Bridges.lean:634); served_tv_mean_le_massweighted (WitCert/Bridges.lean:977);
 served_tv_mean_le_omega_subgaussian (WitCert/Bridges.lean:1049); softmax_nonneg
 (WitCert/Bridges.lean:67); softmax_sum (WitCert/Bridges.lean:73); softmax_tilt
 (WitCert/Bridges.lean:79); softmax_tv_bridge_tanh (WitCert/Bridges.lean:362);
 sqrt_jensen_weighted (WitCert/Bridges.lean:886); sum_exp_pos (WitCert/Bridges.lean:70);
 tv_affine_relax (WitCert/Bridges.lean:144); tv_chord_bound (WitCert/Bridges.lean:235);
 tv_le_hellinger (WitCert/Bridges.lean:445); tv_le_subgaussian (WitCert/Bridges.lean:548);
 tv_le_tanh (WitCert/Bridges.lean:271); var_p_delta_le_trace_cov (WitCert/Bridges.lean:666).